Fitting SMPL to Images: Optimization and Transformers

Lecture 18
Notes for Probabilistic Models of the Visual Cortex

Alan Yuille

Johns Hopkins University

3 November, 2026

What this lecture covers

3D humans from one image

Fitting a generative model

Non-linear least squares

SMPLify

HMR 2.0: transformers

Tracking humans in 4D

Summary

The first four parts follow Pons-Moll's lecture on fitting the SMPL (Skinned Multi-Person Linear) body model to images by optimization (Pons-Moll, P); the last two follow Goel's talk on Humans in 4D (Goel, H), the transformer-based state of the art. A caption citing "P p. *N*" or "H p. *N*" took its picture from page *N* of these slides. Each caption says what its colors mark.

Two ways to estimate 3D humans

The task of this lecture is to take a single RGB image (an ordinary color photograph, with red, green and blue values at each pixel) of a person and to estimate the person's 3D pose and 3D shape: the angles of the joints and the orientation of the body, and the build of the body (height, girth, proportions). In Pons-Moll's words, we want to "see" the person in 3D.

There are two ways to do it. A discriminative model conditions on the image: it is a function, learnt from examples, that maps the image directly to the pose and shape (the name comes from classification, where such models learn the posterior that discriminates between the classes). A generative model explains the image: from a candidate pose and shape it predicts what the image, or features computed from it, should look like, compares the prediction with the actual image, and changes the candidate until the two agree. The generative route comes first here: it is the one that needs a body model, an objective and an optimizer. The transformer models of Goel's talk are discriminative.

Two ways to estimate 3D humans

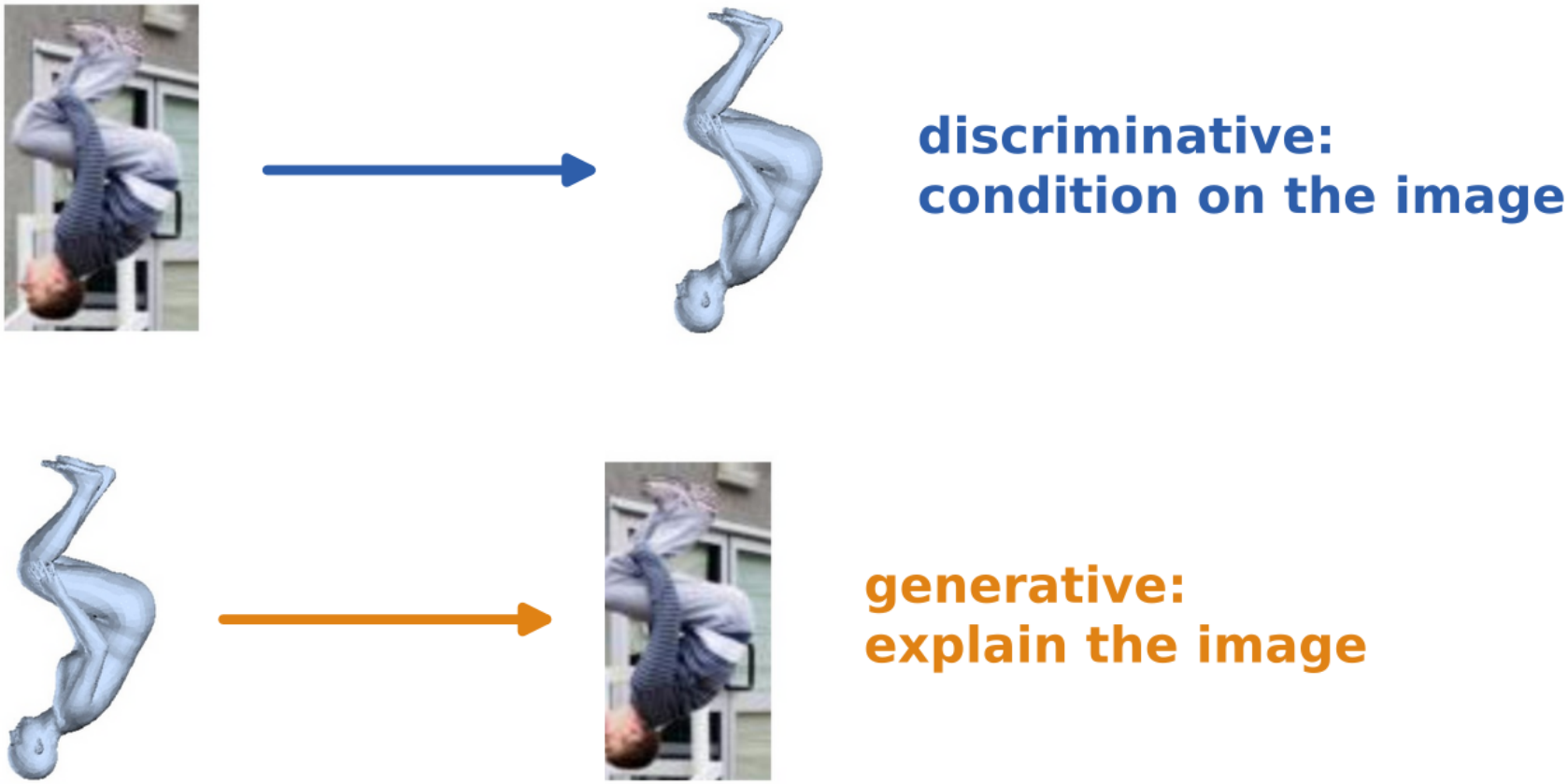


Figure 1: A back flip and the 3D body fitted to it. Blue arrow: a discriminative model maps the image to the body. Orange arrow: a generative model renders the body to predict the image, to be compared with it. P p. 5.

Many degrees of freedom and depth ambiguity make 3D hard

Why is this hard? The body has many degrees of freedom: about two dozen joints, each rotating in three dimensions. Motion is highly dynamic, skinning (the way skin and muscle deform as joints bend) changes the surface, clothing hides it, and outdoor scenes vary without limit. People also move in very individual ways, so the variability of motion patterns is large.

The deepest difficulty is depth ambiguity. A camera projects the 3D world onto a 2D image and loses the distance along each line of sight. Take a limb of known length whose two ends are seen in the image: one end can lie in front of the other or behind it by the same amount and project to the same two points. Under scaled orthographic projection (a camera far from the body) with a given scale, each limb has this two-fold ambiguity, so n limbs give 2^n poses with the same image (Taylor, 2000): many 3D poses produce exactly the same projection.

Many degrees of freedom and depth ambiguity make 3D hard

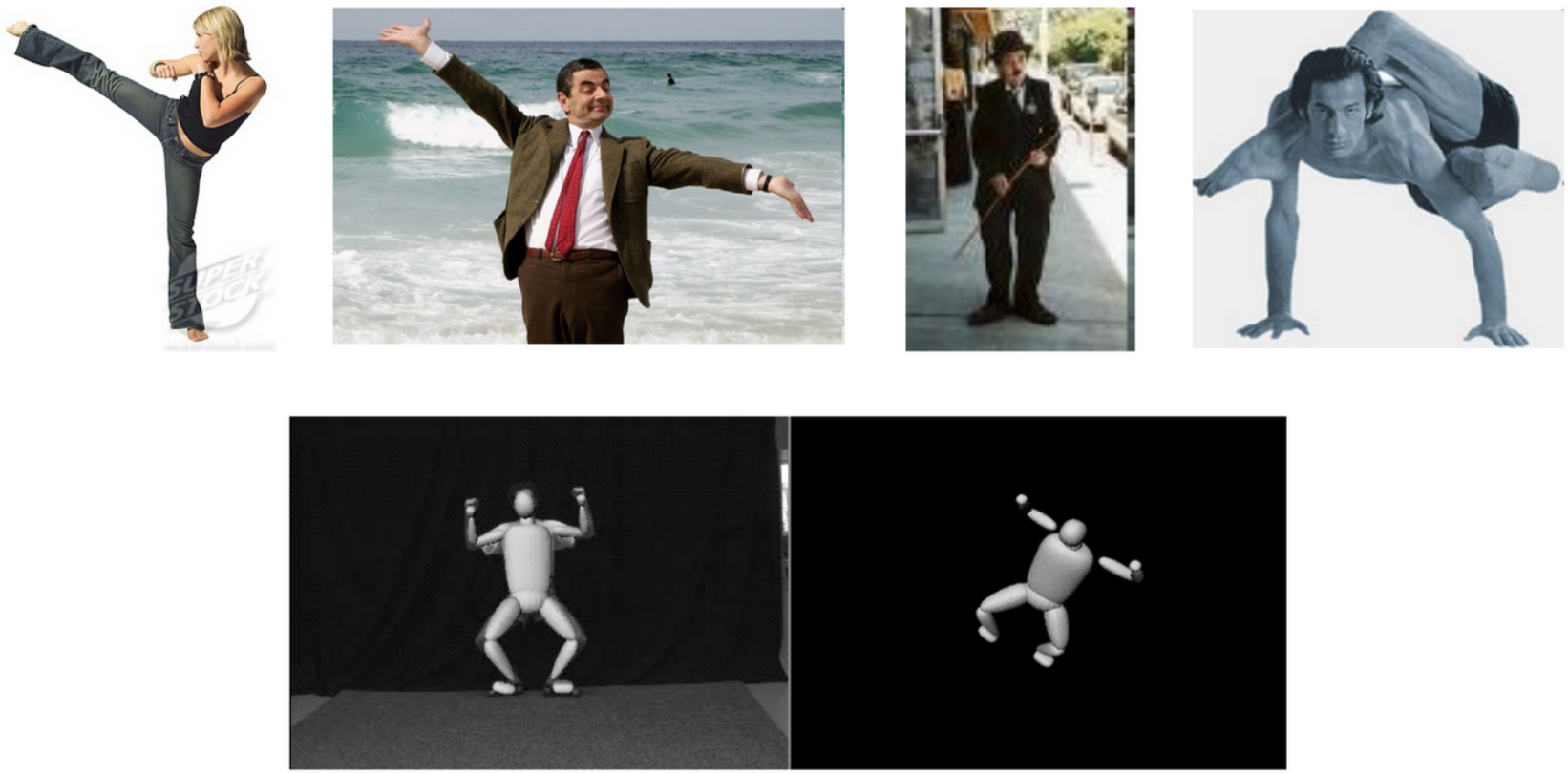


Figure 2: The four photographs: the variability of poses, clothing and settings (P p. 7). The two renderings: a body model of cylinders and ellipsoids seen from the camera, aligned with the person, and the same model seen from another viewpoint, where its limbs are at depths quite unlike the person's (Sminchisescu and Triggs, 2001; P p. 8).

We need a body model

Prior information about humans removes much of the ambiguity. We know that humans have a fixed skeletal structure (bone lengths do not change) and that motion is mostly articulated (body parts move rigidly about their joints). Human shape is roughly symmetric and lives in a subspace: a few numbers describe most of the differences between people. A body model is a parametrized description of the body that builds this knowledge in, so that estimating a person means estimating a few parameters.

Earlier body models came in two kinds. 2D models draw the body parts as rectangles in the image, connected by springs at the joints (pictorial structures). 3D models build the body from simple solids: cylinders, ellipsoids, or Gaussian blobs (the body as a sum of 3D Gaussian densities); or from rigged scans, 3D scans of one person with a skeleton (a rig) embedded so that the surface moves with the bones.

We need a body model

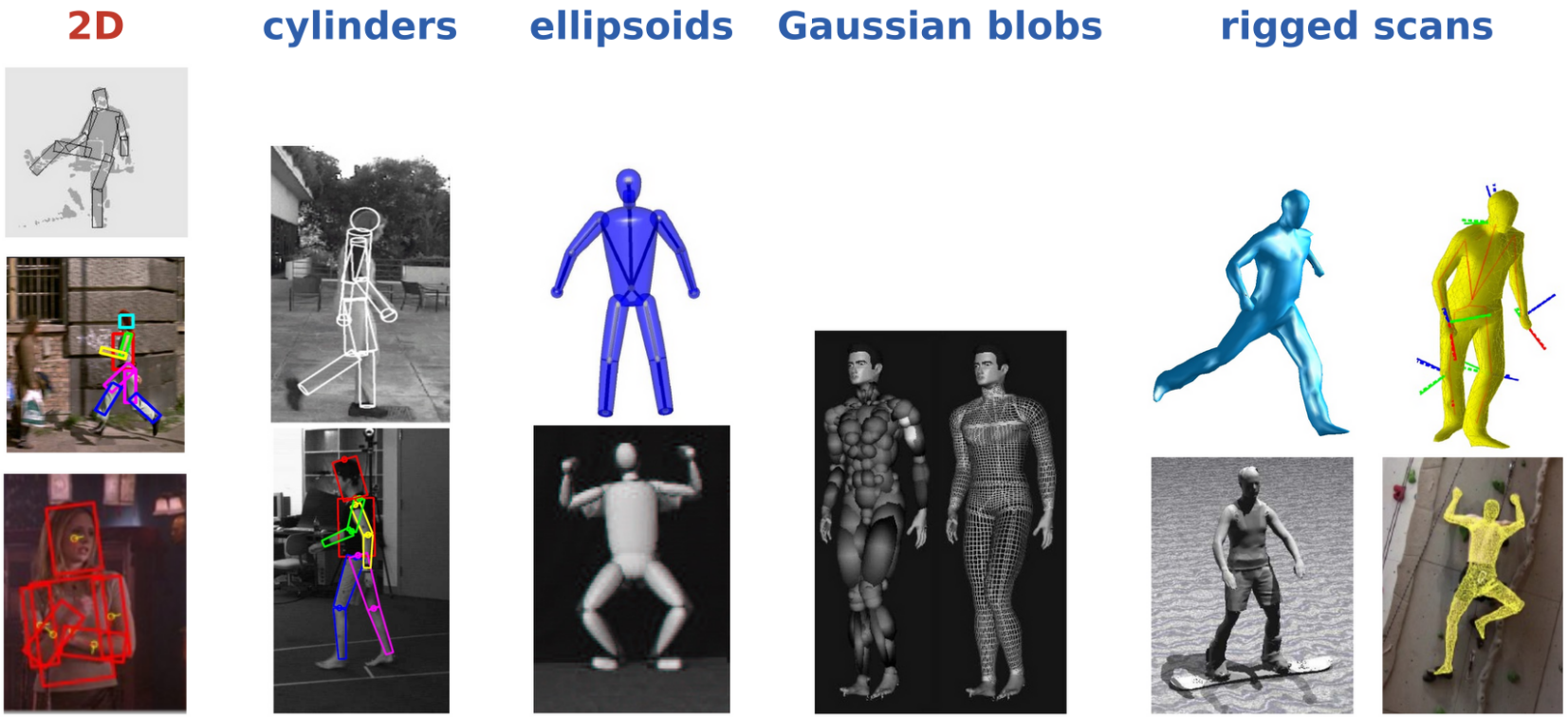


Figure 3: Body models of the past (P p. 11): 2D models (red label), which draw the parts as rectangles in the image, and 3D models (blue labels) made of cylinders, ellipsoids or Gaussian blobs, or rigged scans.

SMPL is the de-facto model for human pose and shape

SMPL, the Skinned Multi-Person Linear model (Loper et al., 2015), is nowadays the de-facto model for estimating human pose and shape from images. It was learnt from thousands of registered 3D scans (brought into vertex-to-vertex correspondence with a template): about 1800 scans of 40 people in many poses, and about 3800 people in one pose.

Its shape parameters $\vec{\beta}$ are the coefficients of a principal component analysis (PCA) of body shapes: each body is written as the mean body plus a weighted sum of a few directions of variation (the principal components), and $\vec{\beta}$ are the weights. SMPLify and the networks of this lecture use 10 of them. Its pose parameters $\vec{\theta}$ are the 3D rotations of 23 joints relative to their parents, plus the rotation of the whole body (the root), 3 numbers each (an axis scaled by the angle): $3 \times 24 = 72$ numbers. Its output is a mesh of 6890 vertices; the model is flexible, easy to use, and adapts to different shapes.

SMPL is the de-facto model for human pose and shape

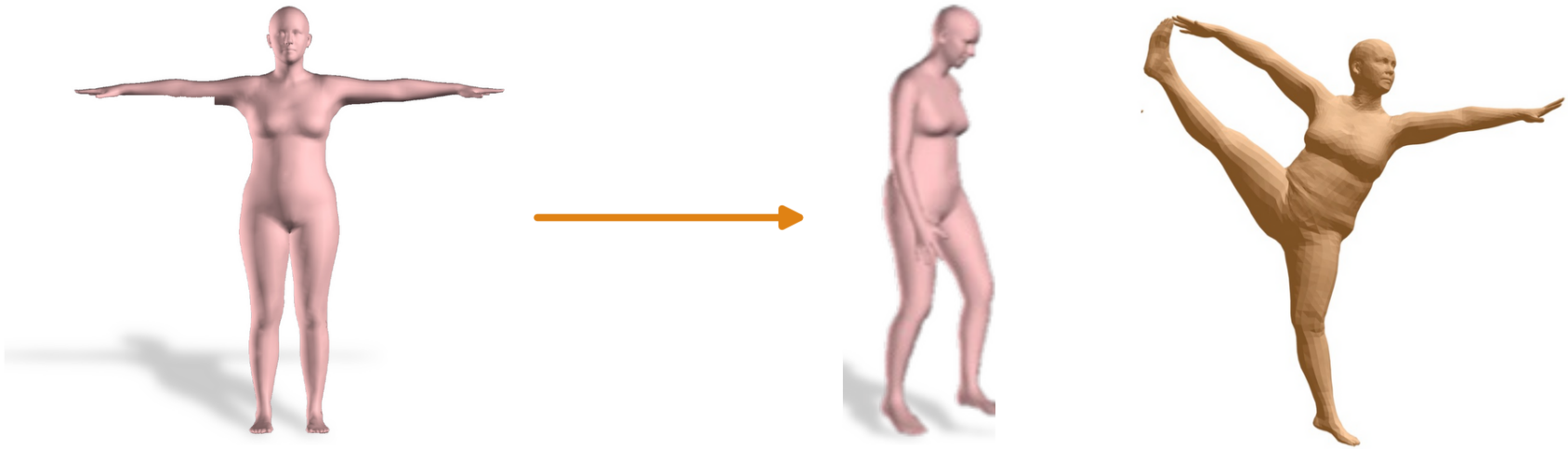


Figure 4: The SMPL template in its rest pose (the T-pose), the template with a pose $\vec{\theta}$ and a shape $\vec{\beta}$ applied, and another pose and shape. P pp. 12, 34.

SMPL deforms a template, then poses it by skinning

SMPL produces a mesh in two steps: it first deforms a template mesh in its rest pose, then poses the deformed mesh. The deformation adds two sets of blend shapes to the template's vertices $\bar{\mathbf{T}}$ (a term from computer graphics: fixed vertex offsets, mixed in with weights):

$$T_P(\vec{\beta}, \vec{\theta}) = \bar{\mathbf{T}} + B_S(\vec{\beta}) + B_P(\vec{\theta}),$$

$$B_S(\vec{\beta}) = \sum_{n=1}^{|\vec{\beta}|} \beta_n \mathbf{S}_n, \qquad B_P(\vec{\theta}) = \sum_{n=1}^{207} \left(R_n(\vec{\theta}) - R_n(\vec{\theta}^*)\right) \mathbf{P}_n.$$

The shape blend shapes B_S move every vertex along the principal components $\mathbf{S}_n$. The pose blend shapes B_P correct the surface for the pose: $R(\vec{\theta})$ lists the nine entries of each of the 23 joint rotation matrices ($9 \times 23 = 207$ numbers), $\vec{\theta}^*$ is the rest pose, and each entry's change moves the vertices along a learnt offset $\mathbf{P}_n$. They vanish in the rest pose.

SMPL deforms a template, then poses it by skinning

The deformed mesh is then posed:

$$M(\vec{\beta}, \vec{\theta}) = W\big(T_P(\vec{\beta}, \vec{\theta}),\, J(\vec{\beta}),\, \vec{\theta},\, \mathcal{W}\big).$$

$J(\vec{\beta})$ are the 3D joint locations, regressed linearly from the vertices of the shaped mesh, so the skeleton follows the body's shape. W is linear blend skinning: each bone k moves rigidly with the rotations of the joints between it and the root, and each vertex i moves by the average of these rigid motions, weighted by its blend weights $w_{k,i}$ (the matrix $\mathcal{W}$; at most four bones influence a vertex). Near a joint a vertex has weight $\frac{1}{2}$ on each bone, and averaging two rotations shrinks the surface. In the figure's 2D arm, bending the elbow by 110° thins the elbow from 0.36 to $0.36 \cos 55^\circ = 0.21$; the pose blend shape of this pose pushes the vertices back out (red arrows), up to 0.077. SMPL learns such offsets for every pose from the registered scans.

SMPL deforms a template, then poses it by skinning

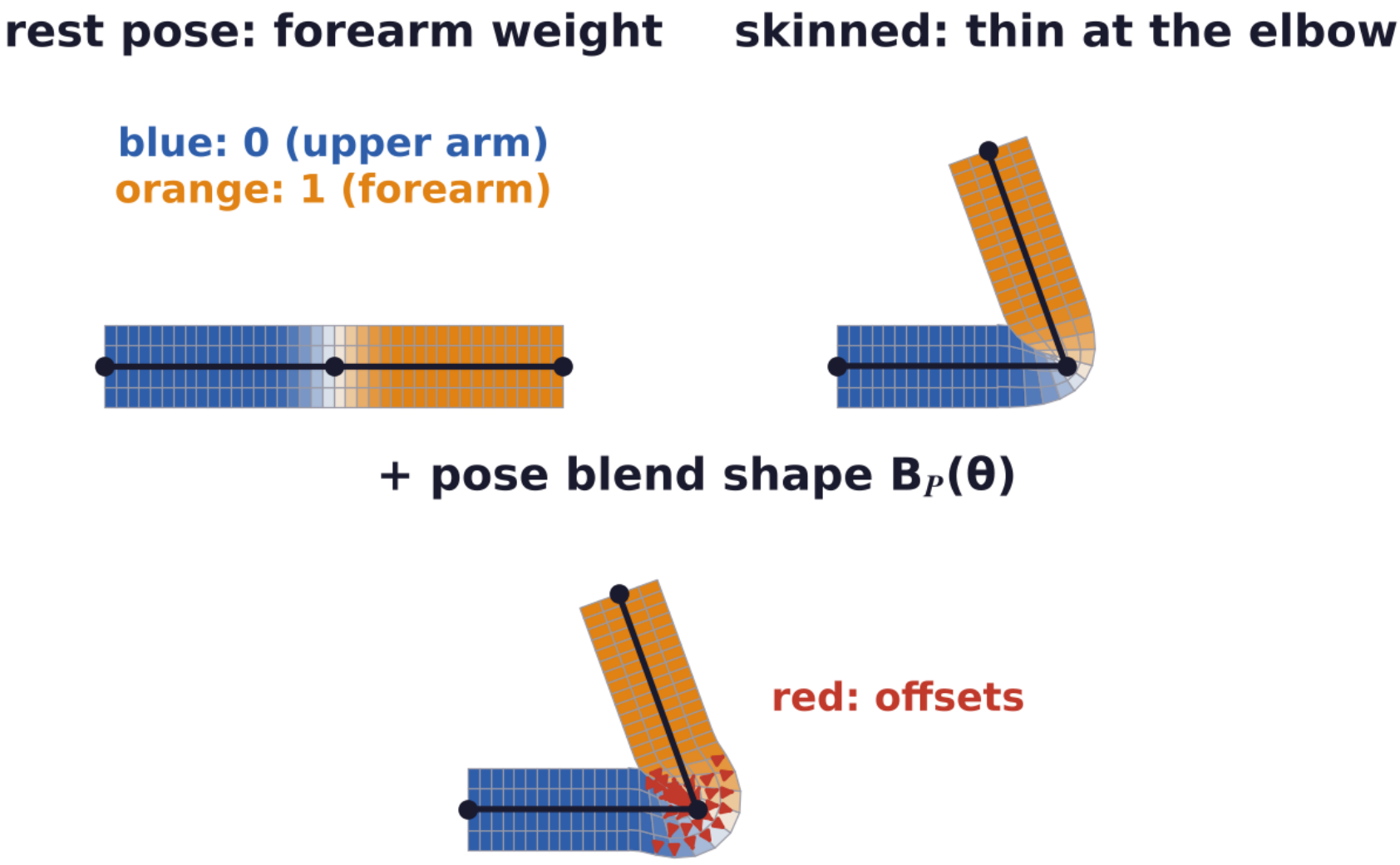


Figure 5: Skinning a 2D arm (dots: joints). Weights on the forearm run from 0 (blue) to 1 (orange); bending the elbow by 110° thins it, and the pose blend shape (red) restores it.

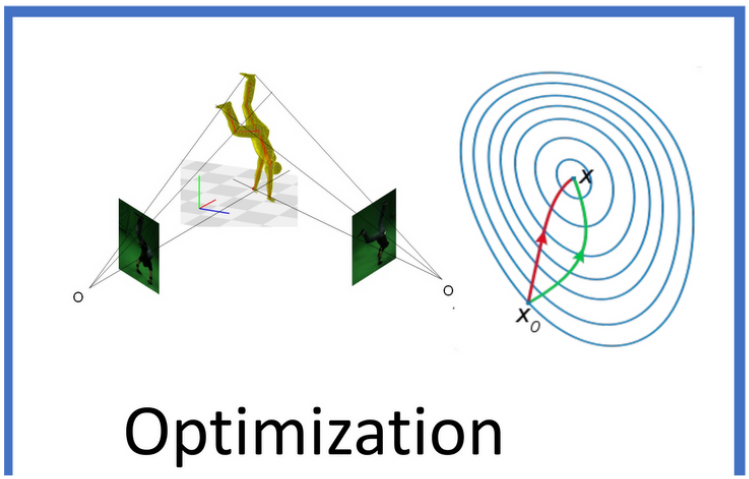
Inference with a generative model such as SMPL

The problem can now be stated. The input is an RGB image I; the unknowns are the model parameters x: the pose $\vec{\theta}$, the shape $\vec{\beta}$ and, optionally, the camera. A generative model supplies the likelihood $p(I \mid x)$, how probable the image is for given parameters, and prior knowledge supplies $p(x)$. Bayes' rule combines them:

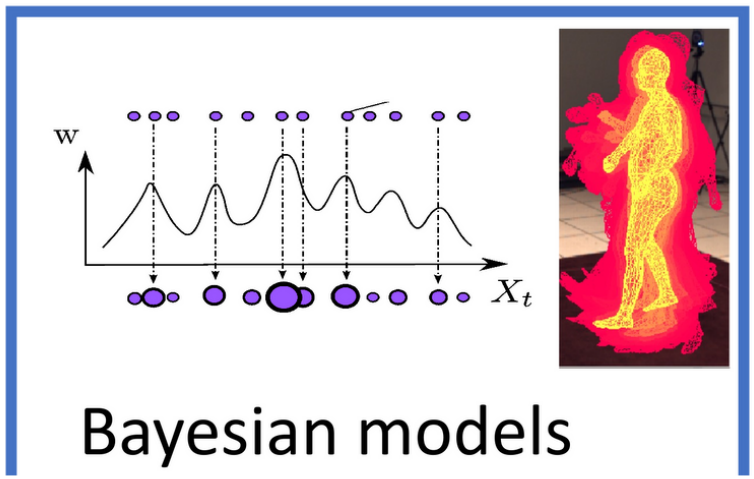
$$\underbrace{p(x \mid I)}_{\text{posterior}} \propto \underbrace{p(I \mid x)}_{\text{likelihood}} \times \underbrace{p(x)}_{\text{prior}}.$$

There are two ways to use the posterior. Optimization looks for its maximum, the MAP (maximum a posteriori) estimate, which is the minimum of the cost $-\log p(x \mid I)$, reached downhill from a start x_0. Bayesian models represent the whole posterior by weighted samples: many candidate parameter values, each with a weight proportional to its posterior probability (as in a particle filter, which tracks such samples from frame to frame). Samples can keep several hypotheses, for instance the two depths of an ambiguous limb; an optimizer keeps one.

Inference with a generative model such as SMPL



the maximum of p(x | I)



p(x | I) by weighted samples

Figure 6: Optimization: a body model seen by two cameras, and the contours of a cost with two optimizer paths (red, green) from a start x_0 to its minimum x. Bayesian models: samples (top row) of a density over a parameter X_t at frame t, the same samples sized by their weight w (bottom row), and a body model drawn over an image. P p. 15.

Optimization: extract features, predict and match, optimize

Pons-Moll organizes fitting by optimization into three steps, each with its own question. First, extract features from the image: what are good features for fitting? Second, predict the same features from the current model and match them with the image's: how should the 3D SMPL model be matched with the 2D image? Third, optimize: what objective should be minimized, and how? SMPLify is one complete set of answers.

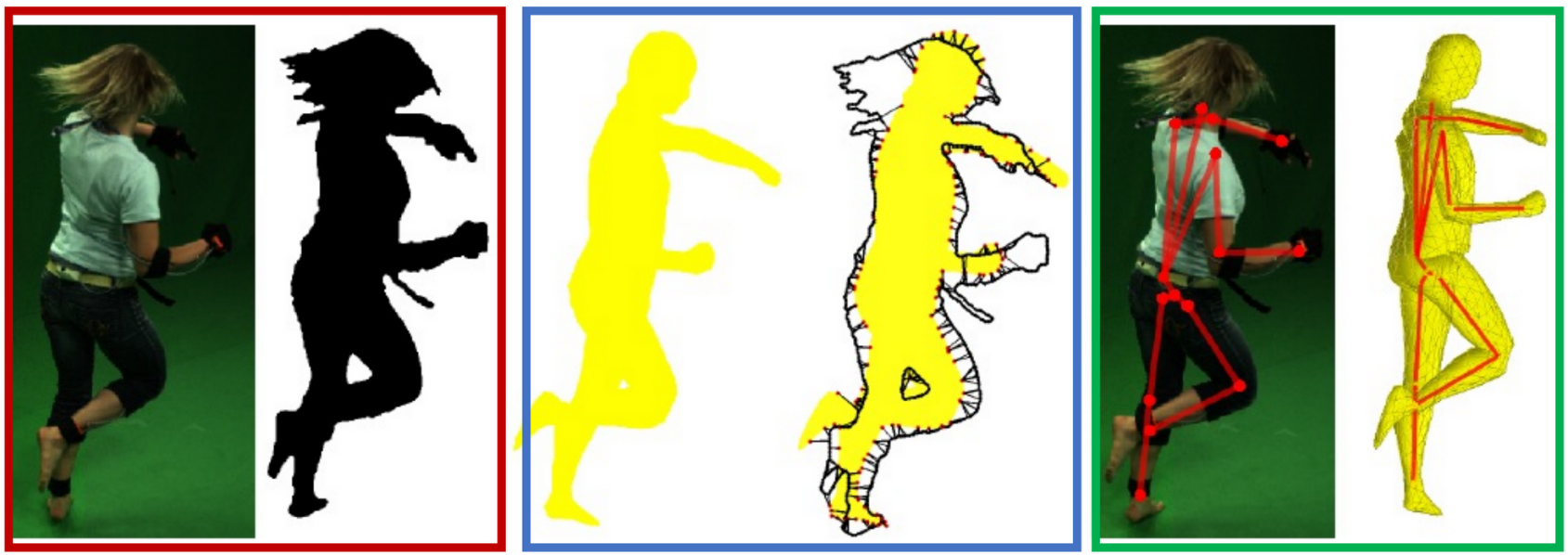


Figure 7: The three steps on one example (P p. 16). Features: the image and its silhouette (black). Matching: the model's silhouette (yellow) and its correspondences (short black lines) with the image's contour (black outline). Optimization: the fitted model over the image and in 3D.

Good features: predicted from the model, fast to compute

A feature is useful for fitting if the model can predict it and it is fast to compute. The silhouette is the set of pixels covered by the person, found by subtracting a known background; edges are the pixels where the intensity changes abruptly. The distance transform of a silhouette gives each pixel its distance to the nearest silhouette pixel: zero inside, growing outside, it turns "is this model point inside the silhouette?" into a smooth cost. SIFT (the scale-invariant feature transform) describes small patches by histograms of gradient directions, to match them across images; optical flow is the apparent motion of each pixel between frames.



Figure 8: Appearance and foreground (image with fitted skeleton and 3D inset; foreground mask), silhouette, edges, and distance transform (blue: near, red: far). P p. 19.

Two distance-transform terms make the fit consistent

Let S_I be the image's silhouette and S_M the model's (the pixels the projected model covers), with distance transforms D_I and D_M, and let q be a pixel. Two costs can compare them:

$$1) \sum_{q \in S_M} D_I(q), \qquad 2) \sum_{q \in S_I} D_M(q).$$

Term 1 sums, over the model's pixels, their distance to the image silhouette: it is zero when the model lies inside the silhouette and pushes the model inside. Term 2 sums, over the image's pixels, their distance to the model: it is zero only when the model covers the whole silhouette, so it forces the model to explain the image. With term 1 alone a model shrunk inside the silhouette costs nothing.

Two distance-transform terms make the fit consistent

A computed example: for an elliptical silhouette of 821 pixels (semi-axes 12 across and 22 up), a model with half its semi-axes, or a single pixel, has term 1 equal to 0, while term 2 is 2971 and 9479; a model that matches exactly scores 0 on both, and one shifted sideways by 6 pixels scores 807 on both. Term 1 is zero exactly when the model lies inside the silhouette, term 2 exactly when it covers it, so their sum is zero only for a model that matches the silhouette. Pons-Moll, after Sminchisescu (2002), calls the pair consistent.



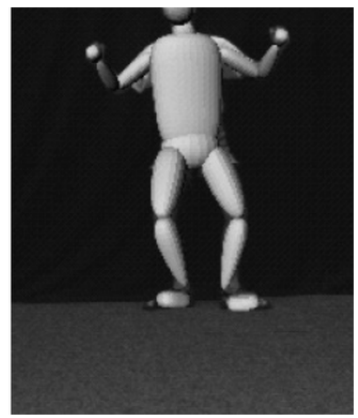


Figure 9: The image's distance transform D_I (dark: near the silhouette), a model fitted with term 1 only, which lies inside the silhouette but does not cover it (inconsistent), and the model fitted with both terms (consistent). P p. 20.

Project the model features and match them to the image

Matching works in a loop (Pons-Moll et al., 2010). Features are extracted from the input image once. At each iteration, the features of the current 3D model are projected into the image by the camera, matched with the image features, and the model is changed to reduce the mismatch.

Matching has a direction. Model to image: every projected model feature looks for its nearest image feature. This fails when the image shows only part of the body, since the hidden parts find wrong matches. Image to model: every image feature looks for its nearest model feature. Then features of the background, or of another person, become outliers that pull the model away. Using both directions, as the two distance-transform terms do, gives a consistent fit, but the squared errors of least squares let a few outliers dominate; robust costs, which grow more slowly for large errors, limit their influence.

Project the model features and match them to the image

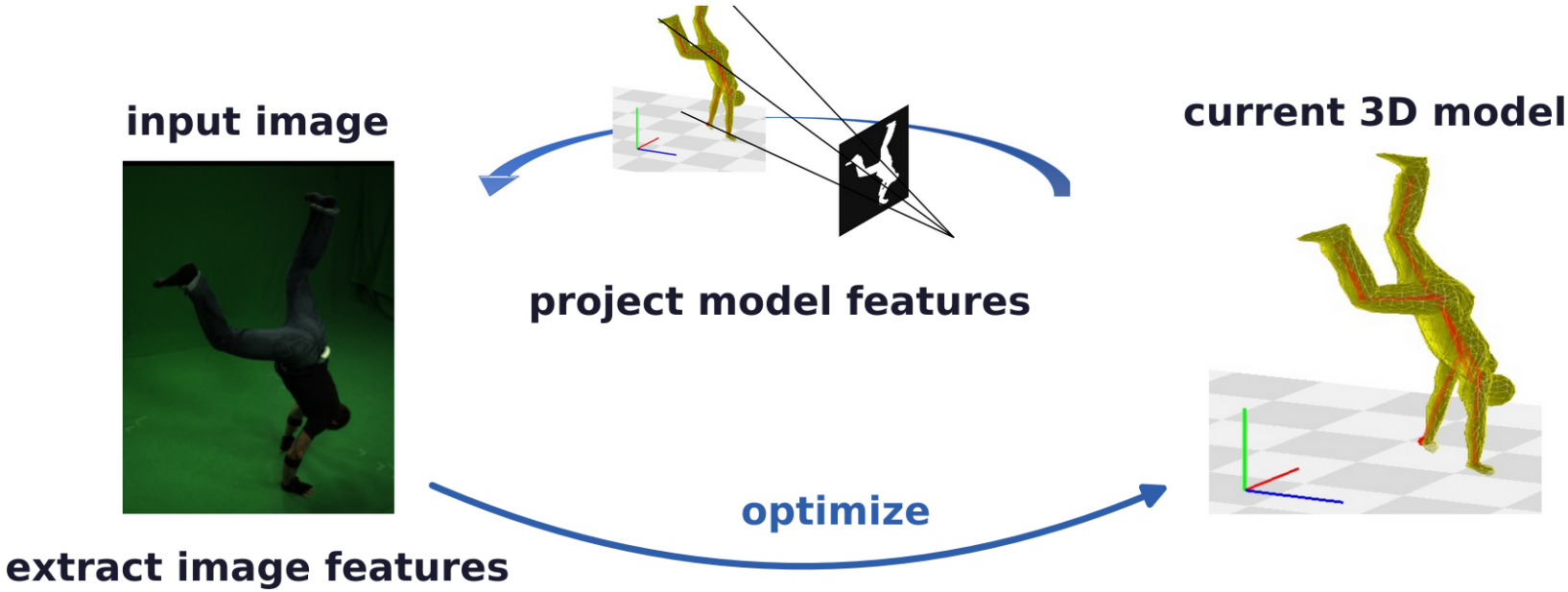


Figure 10: The fitting loop (P p. 22): features are extracted from the input image; the current 3D model's features are projected through the camera onto its image plane (black square); the optimizer (lower blue arc) changes the model to match.

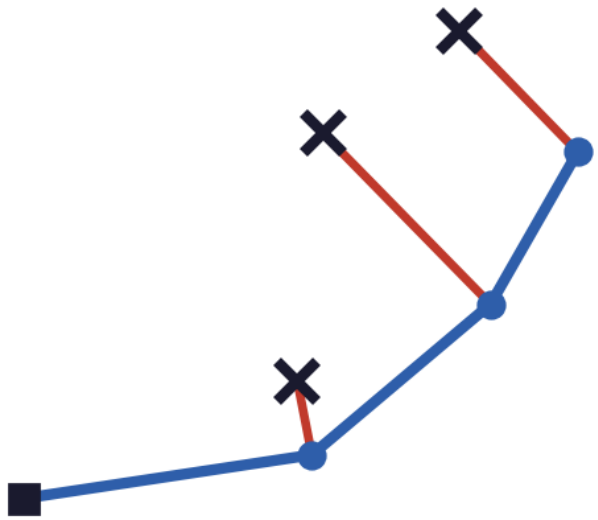
A lot of problems are non-linear least squares

What objective should be optimized? A lot of fitting problems take the form of non-linear least squares. Let $\mathbf{x}_t$ be the SMPL parameters for image t, $\mathbf{r}_i$ the i-th observed 2D image feature and $\tilde{\mathbf{r}}_i(\mathbf{x}_t)$ the same feature predicted by the model. The objective sums the squared distances over all N features:

$$e(\mathbf{x}_t) = \sum_{i=1}^{N} e_i(\mathbf{x}_t) = \sum_{i=1}^{N} \left\| \tilde{\mathbf{r}}_i(\mathbf{x}_t) - \mathbf{r}_i \right\|^2.$$

The problem is non-linear because the predicted features depend on $\mathbf{x}_t$ through rotations, skinning and the camera projection. A toy problem shows everything that follows: a planar arm of three links of lengths 1, 0.8 and 0.6 from a fixed shoulder, whose parameters $\mathbf{x}$ are its three joint angles and whose features are the positions of the ends of its three links. The observed joints are those of the arm at angles $(0.5, 0.9, -0.8)$ radians, plus a little Gaussian noise.

A lot of problems are non-linear least squares



$\tilde{r}_i(x)$: the joints of the arm with joint angles x

r_i: the observed joints

residuals $\tilde{r}_i(x) - r_i$

Figure 11: The toy arm at a guess $\mathbf{x}$ (blue), the observed joints ($\times$) and the residuals $\tilde{\mathbf{r}}_i(\mathbf{x}) - \mathbf{r}_i$ (red); the square is the fixed shoulder.

Least squares is maximum likelihood; a prior makes it MAP

Why squares? Suppose that the observations $\mathbf{y}_t$ of image t (the features $\mathbf{r}_i$) equal the model's predictions plus errors that are independent and Gaussian with variance σ^2 in each coordinate. The likelihood is then a product of Gaussians, and

$$p(\mathbf{x}_t \mid \mathbf{y}_t) \propto p(\mathbf{y}_t \mid \mathbf{x}_t)\, p(\mathbf{x}_t) \propto \exp\Big(-\frac{1}{2\sigma^2} \sum_{i=1}^{N} e_i(\mathbf{x}_t) \Big)\, p(\mathbf{x}_t),$$

$$-\log p(\mathbf{x}_t \mid \mathbf{y}_t) = \frac{1}{2\sigma^2}\, e(\mathbf{x}_t) - \log p(\mathbf{x}_t) + \text{const.}$$

Minimizing $e(\mathbf{x}_t)$ alone maximizes the likelihood: least squares is the maximum likelihood estimate. Adding $-\log p(\mathbf{x}_t)$ to the objective gives the MAP estimate; this is how the priors on pose and shape of SMPLify enter. (Pons-Moll's slide writes e_i^2 in the exponent; e_i is already squared, so the exponent is $-\sum_i e_i/(2\sigma^2)$.) The same reading explains robust costs: a cost that grows slowly for large errors corresponds to an error distribution with heavier tails than the Gaussian, one that expects occasional outliers.

Linearizing the residuals gives the Gauss-Newton step

To minimize e, write it in vector form. Stack the 2D residuals $\mathbf{e}_i = \tilde{\mathbf{r}}_i(\mathbf{x}_t) - \mathbf{r}_i$ of the N matches into one vector $\mathbf{e} = (\mathbf{e}_1^\mathsf{T}, \ldots, \mathbf{e}_N^\mathsf{T})^\mathsf{T} \in \mathbb{R}^{2N}$, so that $e(\mathbf{x}_t) = \mathbf{e}^\mathsf{T}\mathbf{e}$; each term e_i of the sum is the squared length of $\mathbf{e}_i$. (Pons-Moll writes e_i for both; the bold vector and the plain number are kept apart here.)

Let $\mathbf{J}_t$ be the Jacobian of the vector $\mathbf{e}$ at $\mathbf{x}_t$: the $2N \times d$ matrix of the derivatives of each residual coordinate with respect to each of the d parameters. Each pair of its rows belongs to one feature, often a model vertex (how it moves when a parameter changes). To first order, a step $\Delta\mathbf{x}$ changes the residuals to $\mathbf{e}(\mathbf{x}_t + \Delta\mathbf{x}) \approx \mathbf{e} + \mathbf{J}_t\Delta\mathbf{x}$, and half the error becomes a quadratic function of the step:

$$\tfrac{1}{2}\,(\mathbf{e} + \mathbf{J}_t\Delta\mathbf{x})^\mathsf{T}(\mathbf{e} + \mathbf{J}_t\Delta\mathbf{x}) = \tfrac{1}{2}\,\mathbf{e}^\mathsf{T}\mathbf{e} + \Delta\mathbf{x}^\mathsf{T} \underbrace{\mathbf{J}_t^\mathsf{T}\mathbf{e}}_{\text{gradient}} + \tfrac{1}{2}\,\Delta\mathbf{x}^\mathsf{T} \underbrace{\mathbf{J}_t^\mathsf{T}\mathbf{J}_t}_{\sim\text{Hessian}} \Delta\mathbf{x}.$$

Linearizing the residuals gives the Gauss-Newton step

The middle term is linear in the step, and $\mathbf{J}_t^{\mathsf{T}}\mathbf{e}$ is exactly the gradient of $\frac{1}{2}e$ at $\mathbf{x}_t$. The last term is quadratic, and $\mathbf{J}_t^{\mathsf{T}}\mathbf{J}_t$ approximates the Hessian (the matrix of second derivatives) of $\frac{1}{2}e$: the full Hessian adds $\sum_k (\mathbf{e})_k \nabla^2 (\mathbf{e})_k$ over the $2N$ coordinates of $\mathbf{e}$, terms small when the residuals are small or nearly linear, and which Gauss-Newton drops. Setting the gradient of the quadratic with respect to $\Delta\mathbf{x}$ to zero gives the Gauss-Newton step:

$$\mathbf{J}_t^{\mathsf{T}}\mathbf{J}_t\,\Delta\mathbf{x} = -\mathbf{J}_t^{\mathsf{T}}\mathbf{e}, \qquad \mathbf{x}_t \leftarrow \mathbf{x}_t + \Delta\mathbf{x}.$$

The step needs only first derivatives, and near the minimum it converges almost as fast as Newton's method, which uses the exact Hessian. Far from the minimum the linearization is poor, and the step can overshoot: on the toy arm from the start $(-2, 1.5, 2)$, the first Gauss-Newton step raises e from 17.2 to 21.5 before the method recovers.

Levenberg-Marquardt damps the Gauss-Newton step

Levenberg (1944) and Marquardt (1963) add a damping term $\mu \geq 0$:

$$\Delta \mathbf{x} = -\left(\mathbf{J}_t^{\mathsf{T}} \mathbf{J}_t + \mu \mathbf{I}\right)^{-1} \mathbf{J}_t^{\mathsf{T}} \mathbf{e}, \qquad \mathbf{x}_t \leftarrow \mathbf{x}_t + \Delta \mathbf{x}.$$

With $\mu = 0$ this is the Gauss-Newton step. For a large μ the matrix is close to $\mu \mathbf{I}$, and the step is close to $-\mathbf{J}_t^{\mathsf{T}} \mathbf{e}/\mu$: a short step down the gradient. The damping is adapted as the method runs: μ is lowered after a step that reduces $e(\mathbf{x})$, so the method behaves like Gauss-Newton where the linearization is good, and raised (and the step tried again) after a step that does not, so it behaves like gradient descent where it is poor. The error therefore never increases. On the toy arm, with μ starting at 1 and changed by factors of 10, Levenberg-Marquardt goes from 17.2 to 0.0029 in five iterations, Gauss-Newton needs six, including its bad first step, and gradient descent with a fixed step of 0.15 is still at 0.37 after twelve. The value 0.0029 is the minimum of e, the part of the noise that no pose of the arm can fit: the best any method can do.

Levenberg-Marquardt damps the Gauss-Newton step

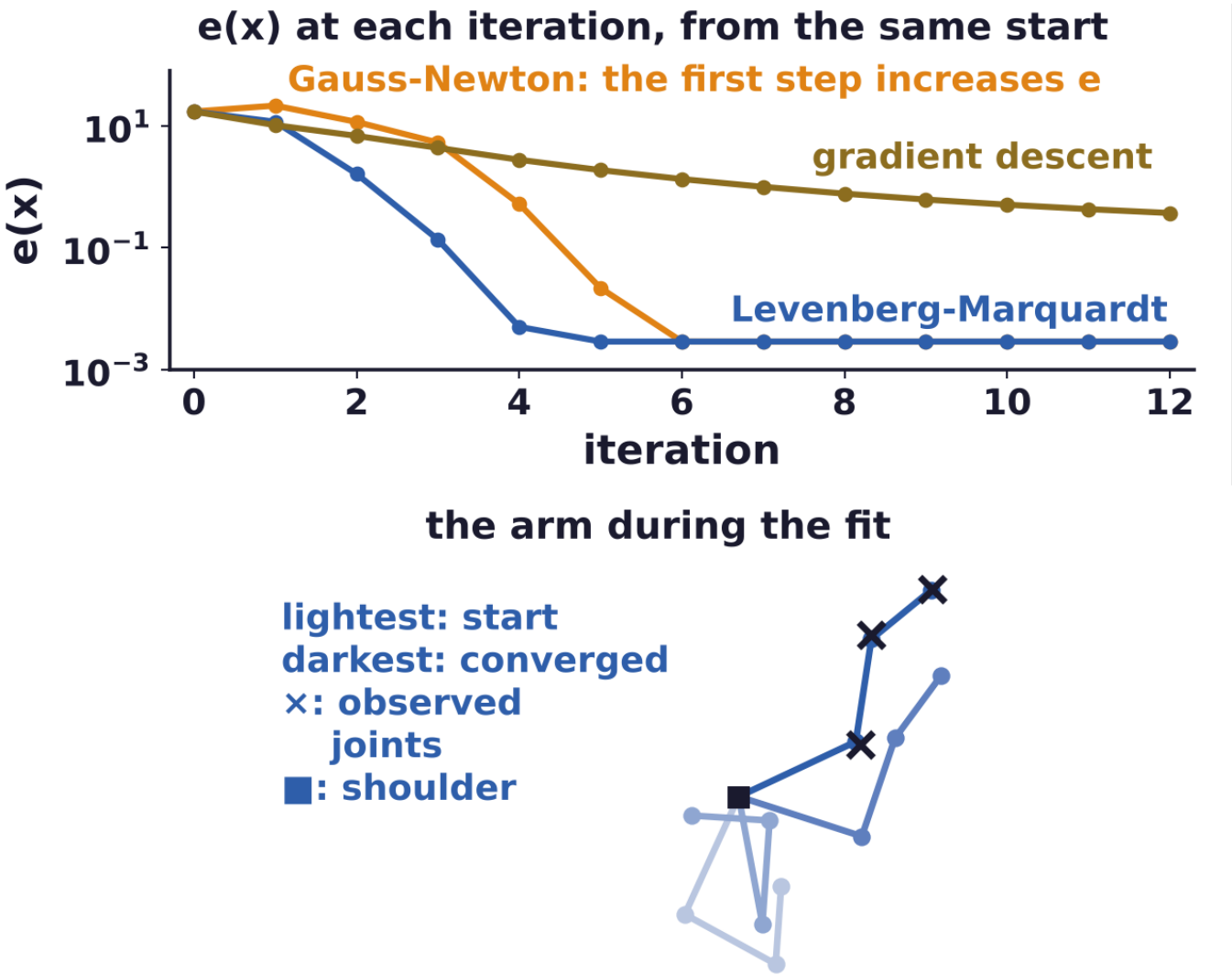


Figure 12: Plot: the error $e(\mathbf{x})$ at each iteration (logarithmic scale) for Gauss-Newton, Levenberg-Marquardt and gradient descent (olive). Arm: the Levenberg-Marquardt fit, from the start (lightest) to the end (darkest); $\times$: observed points.

Covariance scaled sampling tracks a body with one camera

An example from 2001 shows the whole framework (Sminchisescu and Triggs, 2001). A body model of 30 or 34 degrees of freedom, with limbs made of superquadrics (deformable ellipsoids), is tracked in a video from one camera. The features are motion boundaries, edges and optical flow. For each model feature i, $r_i(x)$ is its predicted position and $\bar{r}_i$ the image feature matched to it (the reverse of Pons-Moll's notation), and $\Delta r_i = \bar{r}_i - r_i(x)$. The posterior and the cost of each feature are

$$p(x \mid \bar{r}) \propto \exp\Big(-\sum_i e_i(x) \Big)\, p(x),$$

$$e_i(x) = \begin{cases} \frac{1}{2}\rho_i\big(\Delta r_i\, W_i\, \Delta r_i^{\mathsf{T}}\big) & \text{if matched} \\ \nu & \text{if back-facing} \\ k\nu,\ \ k > 1 & \text{if occluded.} \end{cases}$$

W_i is a positive definite weighting matrix and ρ_i a robust cost. A model feature that faces away from the camera, or that another part hides, has no image feature and pays a constant: ν is a strength related to how often features are outliers.

Covariance scaled sampling tracks a body with one camera

The cost has many local minima; a third of its directions, those that move parts in depth, are almost unconstrained. The method samples from the fit's covariance (the inverse Hessian), inflated along them, and refines each sample within joint limits. Remarkable for 2001, it was limited to lab settings and to easy poses and backgrounds.

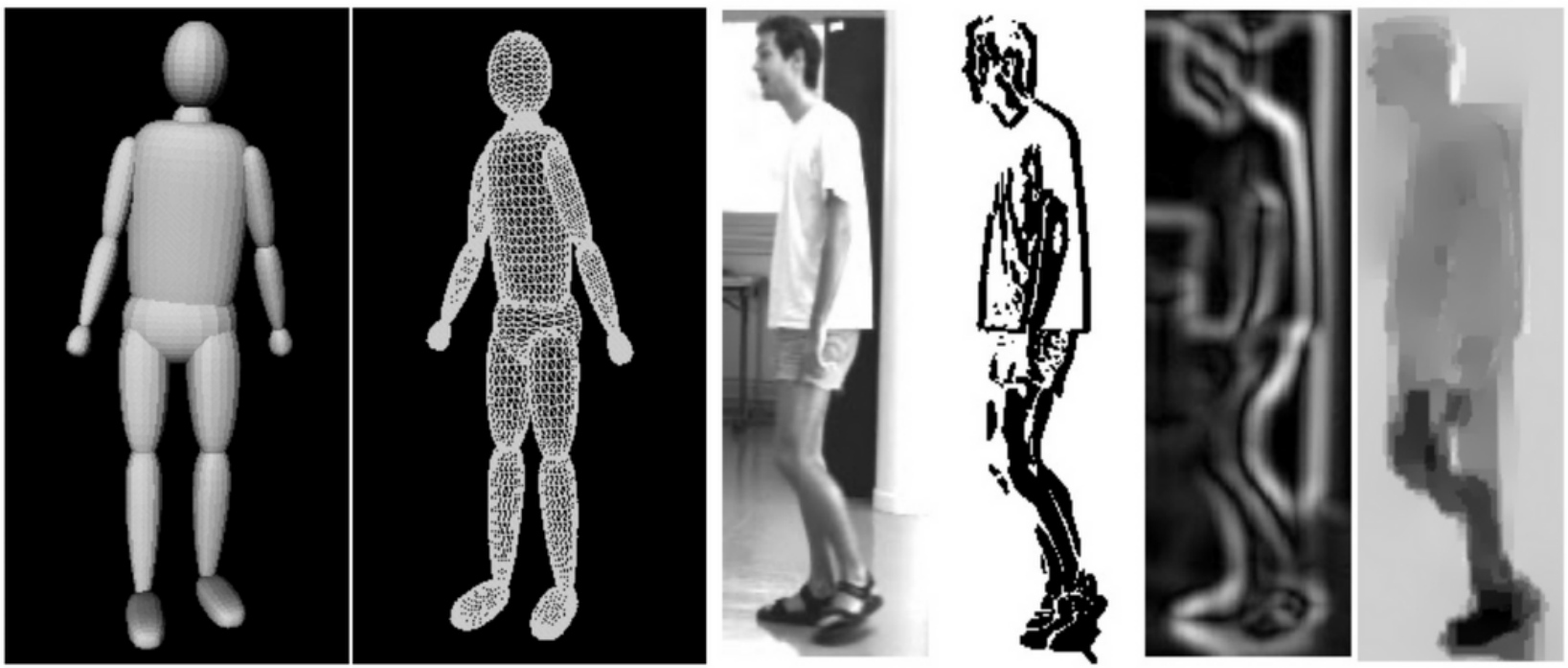


Figure 13: The model shaded and as a mesh, an image, and three feature maps. P p. 30.

What changed since 2001: 2D pose detection and SMPL

Two things changed by 2016. First, 2D pose detection works on most images: a CNN (convolutional neural network), trained on photographs with hand-labelled joints, finds the body's 2D joints, each with a confidence. 2D joints are a very good feature: SMPL predicts them, since its joints $J(\vec{\beta})$ are regressed from its mesh, and the detectors work in the wild (on everyday photographs outside the lab). Second, SMPL appeared. SMPLify (Bogo et al., 2016) combines the two.



Figure 14: 2D joints found by a CNN on images in the wild (colored lines join the joints of each limb). P p. 32.

Bottom up and top down should match

SMPLify runs Pons-Moll's three steps as follows. Bottom up, from the image: a CNN (DeepCut; Pishchulin et al., 2016) detects the estimated 2D joints $J_{est,i}$ with confidences w_i. Top down, from the model: the SMPL joints $J(\vec{\beta})_i$, moved to their posed positions by $R_{\vec{\theta}}$ (the rotations of the skeleton for pose $\vec{\theta}$), are projected by Π_K with parameters K (a perspective camera with known focal length and an unknown translation). The data term compares the two:

$$E_J(\vec{\beta}, \vec{\theta}; K, J_{est}) = \sum_{\text{joint } i} w_i \, \rho\Big(\Pi_K\big(R_{\vec{\theta}}(J(\vec{\beta})_i)\big) - J_{est,i}\Big).$$

ρ is the Geman-McClure penalty, a robust version of the squared distance: in the code of its successor SMPLify-X, $\rho(r) = \sigma^2 r^2/(\sigma^2 + r^2)$ for each coordinate, with $\sigma = 100$ pixels. It grows like r^2 for small errors and levels off at σ^2, so a badly detected joint costs at most a fixed amount. $J(\vec{\beta})$ are joints, unlike the Jacobian $\mathbf{J}_t$ of least squares.

Bottom up and top down should match

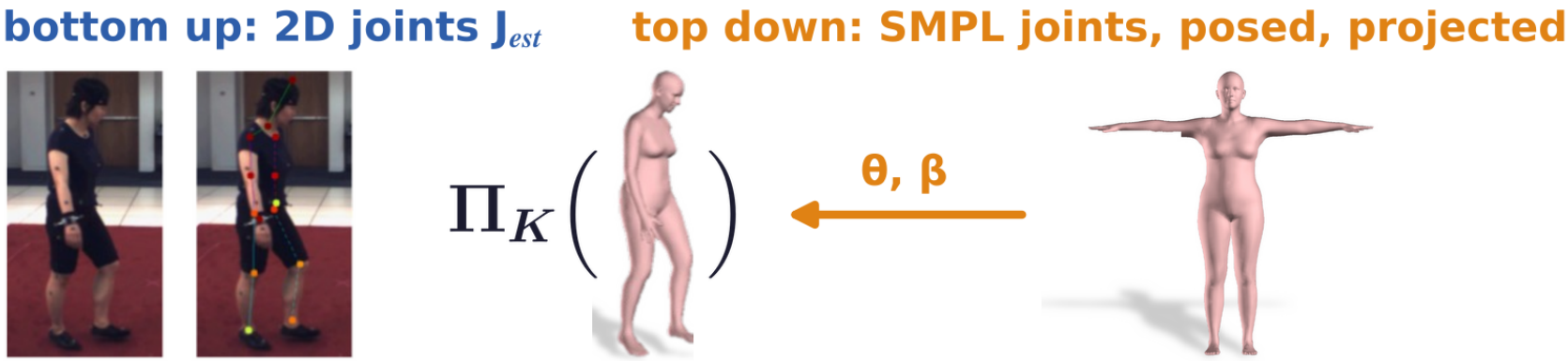


Figure 15: Bottom up: an image and its detected 2D joints. Top down: the SMPL template, posed by $\vec{\theta}$ and shaped by $\vec{\beta}$ (orange arrow), then projected by Π_K. P p. 34.

The data term alone leaves a depth ambiguity

The data term compares 2D joints only, and many 3D poses and shapes project to the same 2D joints. An earlier fit (Guan et al., 2009) aligns with the image and is wrong in 3D: seen from the side, its legs bend as if sitting. Priors choose the plausible bodies among those that fit.

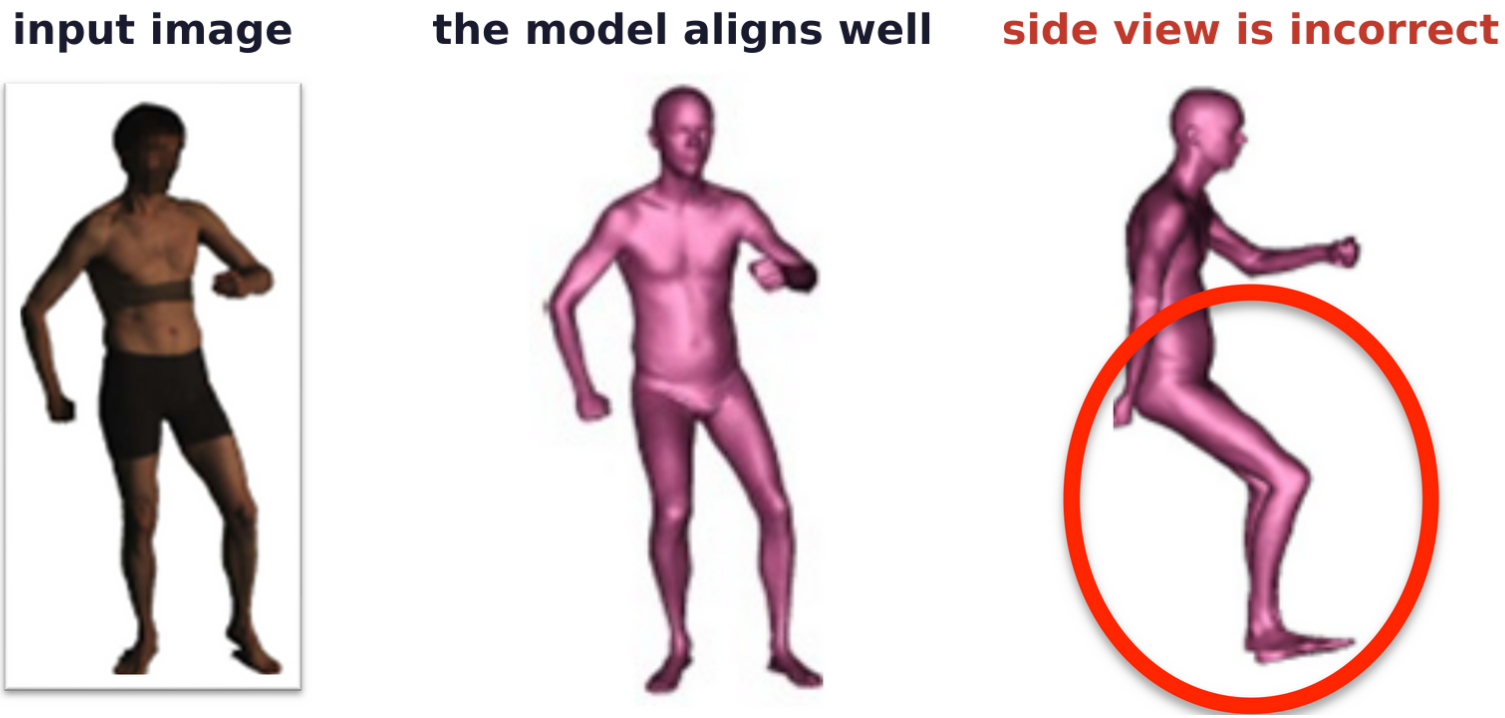


Figure 16: An input image and the fit seen from the camera and from the side (red circle: legs bent). Guan et al. (2009); P p. 37.

Pose and shape priors favor plausible bodies

SMPLify adds three priors on pose and shape. The first penalizes elbows and knees that bend the unnatural way:

$$E_a(\vec{\theta}) = \sum_i \exp(\theta_i),$$

where i runs over the rotations that bend the elbows and knees, with the sign chosen so that natural bending is negative and costs little, while unnatural bending, $\theta_i > 0$, grows exponentially. The second is a prior over whole poses, a mixture of 8 Gaussians fitted to about 1 million poses obtained by fitting SMPL to motion-capture data (the CMU database):

$$E_\theta(\vec{\theta}) = -\log \sum_j g_j \mathcal{N}(\vec{\theta}; \mu_{\theta,j}, \Sigma_{\theta,j}) \approx \min_j \left(-\log(c\, g_j \mathcal{N}(\vec{\theta}; \mu_{\theta,j}, \Sigma_{\theta,j})) \right),$$

with weights g_j, means $\mu_{\theta,j}$ and covariances $\Sigma_{\theta,j}$. The log of a sum is awkward for a least-squares solver, so the sum is replaced by its largest term; c is a positive constant the solver needs.

Pose and shape priors favor plausible bodies

The third prior keeps the shape near the population mean:

$$E_\beta(\vec{\beta}) = \vec{\beta}^{\mathsf{T}} \Sigma_\beta^{-1} \vec{\beta},$$

where Σ_β is diagonal, with the variances of the principal components of the body shapes. It is the squared Mahalanobis distance of $\vec{\beta}$ from the mean shape, which is $-2\log$ of a Gaussian prior up to a constant.

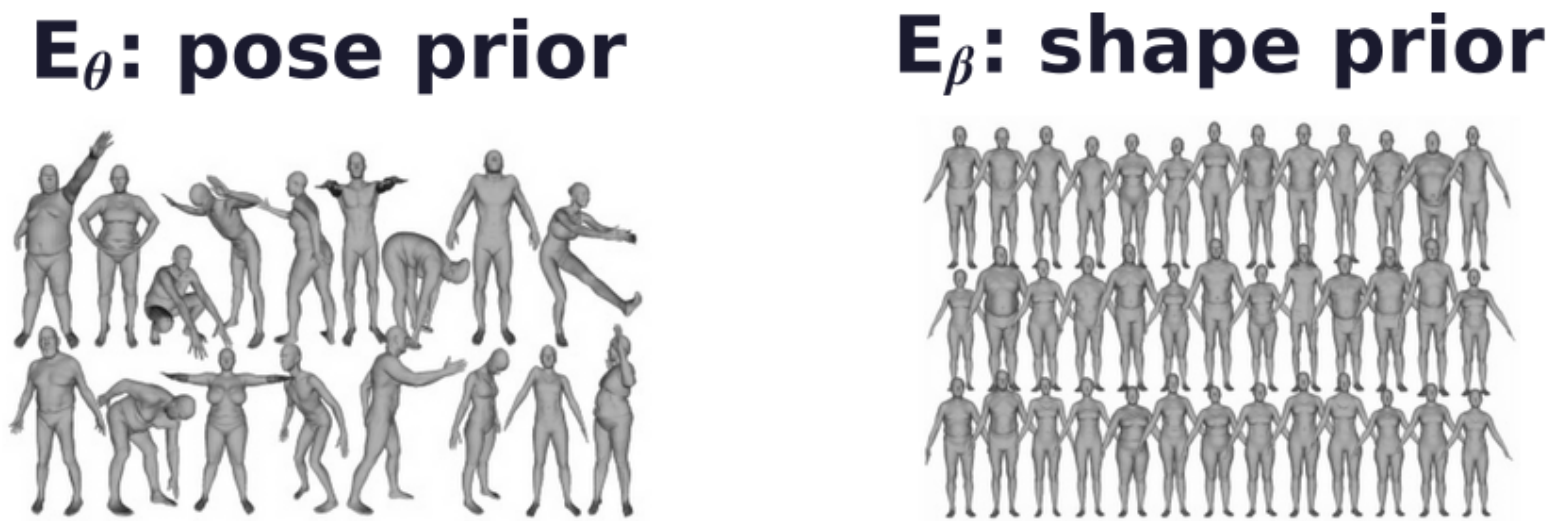


Figure 17: Samples of the poses in the data of the pose prior E_θ, and of the shapes of the shape prior E_β. P p. 38.

Capsules approximate the surface to penalize intersections

A fourth prior is needed: joints alone allow an arm to pass through the body, the 2D joints fit and the 3D body interpenetrates. SMPLify approximates the body by 20 capsules (cylinders with round ends), one per body part except fingers and toes, whose radii and axis lengths are regressed from the shape $\vec{\beta}$. Each capsule is replaced by spheres with centers $C_i(\vec{\theta}, \vec{\beta})$, each treated as a 3D Gaussian of width $\sigma_i(\vec{\beta})$, a third of the capsule's radius, and pairs of spheres that must not meet are penalized:

$$E_{sp}(\vec{\theta}; \vec{\beta}) = \sum_i \sum_{j \in I(i)} \exp\left(- \frac{\| C_i(\vec{\theta}, \vec{\beta}) - C_j(\vec{\theta}, \vec{\beta}) \|^2}{\sigma_i^2(\vec{\beta}) + \sigma_j^2(\vec{\beta})} \right),$$

where $I(i)$ are the spheres incompatible with sphere i. The penalty is largest when two such spheres coincide and falls as they separate.

Capsules approximate the surface to penalize intersections

The paper prints the exponent without its minus sign, a typo: the penalty must fall with the distance. The term is not used to optimize the shape, which it would bias toward thin bodies.

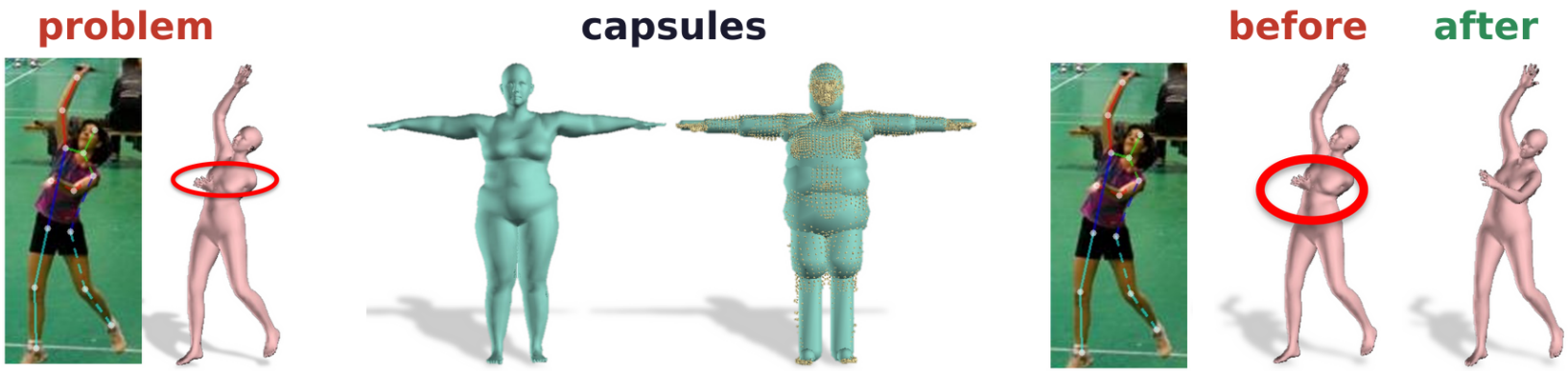


Figure 18: An image and a fit whose arm passes through the body (red ellipse), the body and its capsules, and a fit before and after the penalty. P pp. 40–41.

The SMPLify objective adds four priors to the data term

The full objective adds the four priors to the data term with scalar weights λ:

$$E(\vec{\beta},\vec{\theta}) = E_J(\vec{\beta},\vec{\theta};K,J_{est}) + \lambda_\theta E_\theta(\vec{\theta}) + \lambda_a E_a(\vec{\theta}) + \lambda_{sp} E_{sp}(\vec{\theta};\vec{\beta}) + \lambda_\beta E_\beta(\vec{\beta}).$$

In the Bayesian reading of least squares, the data term plays the role of $-\log p(I \mid x)$ and the priors that of $-\log p(x)$, so minimizing E is a MAP estimate. The optimization is careful about its start. The focal length is known; the camera's depth is estimated by similar triangles from the torso length of the mean shape, assuming the person stands parallel to the image plane; then the camera translation and the body orientation are fitted to the torso joints alone. The full objective is minimized in stages, starting with large λ_θ and λ_β and lowering them, which helps avoid local minima. When the 2D shoulders are close together (the person may face sideways), a second fit rotated by 180° is tried and the one with the lower E_J kept. The solver is Powell's dogleg method, a relative of Levenberg-Marquardt that limits each step to a region where the linearization is trusted; a fit takes less than a minute per image.

The SMPLify objective adds four priors to the data term

SMPLify was judged by eye on the Leeds Sports Pose (LSP) dataset, which lacks 3D ground truth. On HumanEva and Human3.6M (lab videos with motion capture) its 3D error over 14 joints, after Procrustes alignment (best rotation, translation and scale), was 79.9 and 82.3 mm.

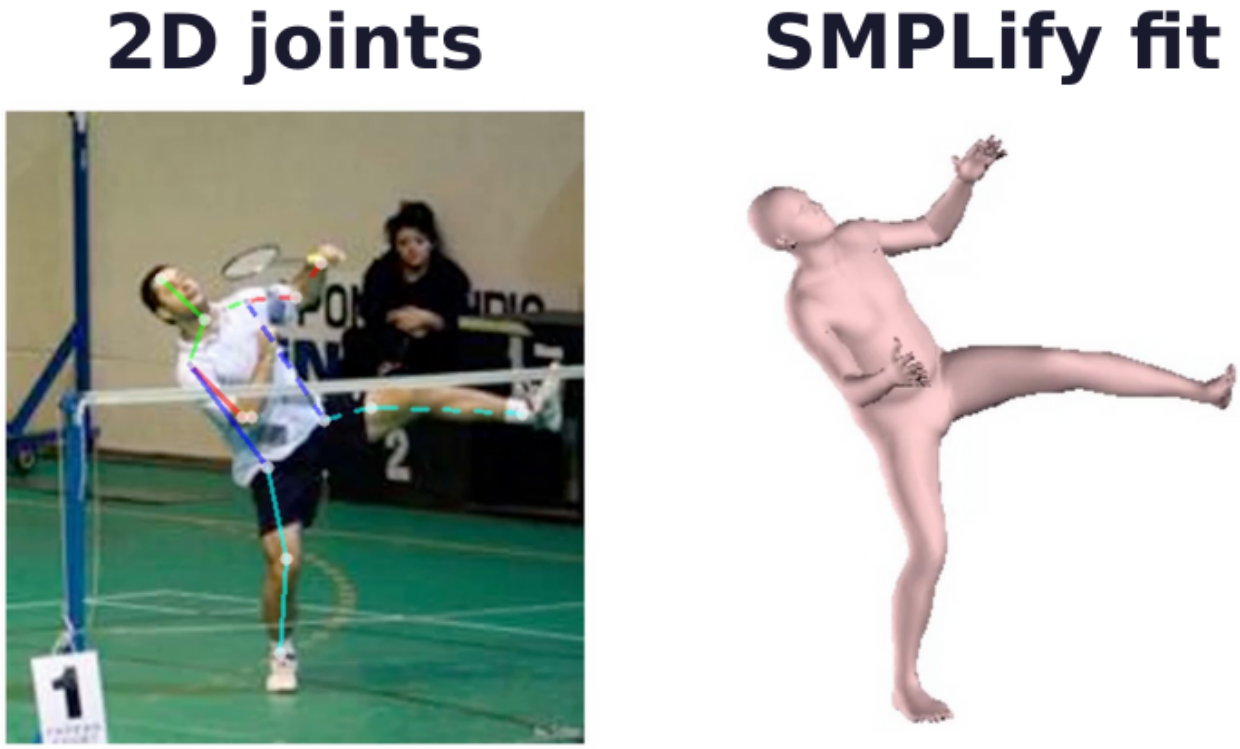


Figure 19: An LSP image with its detected 2D joints, and the SMPLify fit. P p. 43.

SMPLify-X adds hands, face and a learned pose prior

SMPLify-X (Pavlakos et al., 2019) improves on SMPLify with the same key ideas. Its model, SMPL-X, is SMPL with the MANO hand model (articulated hands with finger joints) and the FLAME face model (head, jaw and facial expressions): 10 475 vertices and 54 joints. Its prior on the body pose θ_b (the body's joint rotations, without hands, face and the global rotation) is learnt by a VAE (variational auto-encoder), a network that encodes each pose into a 32-dimensional latent vector $\mathbf{z}$ and decodes it back, trained so that the latent vectors of real poses follow a standard Gaussian. The fit optimizes $\mathbf{z}$ instead of the pose, with the prior

$$E_{\theta_b}(\theta_b) = \|\mathbf{z}\|_2^2, \qquad \theta_b = \text{Decoder}(\mathbf{z}), \qquad \mathbf{z} \in \mathbb{R}^{32},$$

so that any $\mathbf{z}$ near the origin decodes to a plausible pose. A classifier predicts gender from the image and selects the male, female or gender-neutral model. The 2D keypoints of a detector (OpenPose) now include hands, face and feet, interpenetration is detected on the full mesh, and the optimizer is L-BFGS (a quasi-Newton method) in PyTorch, at least 8 times faster than the same fit in SMPLify's older software. On Human3.6M the error drops from 82.3 to 75.9 mm.

SMPLify-X adds hands, face and a learned pose prior

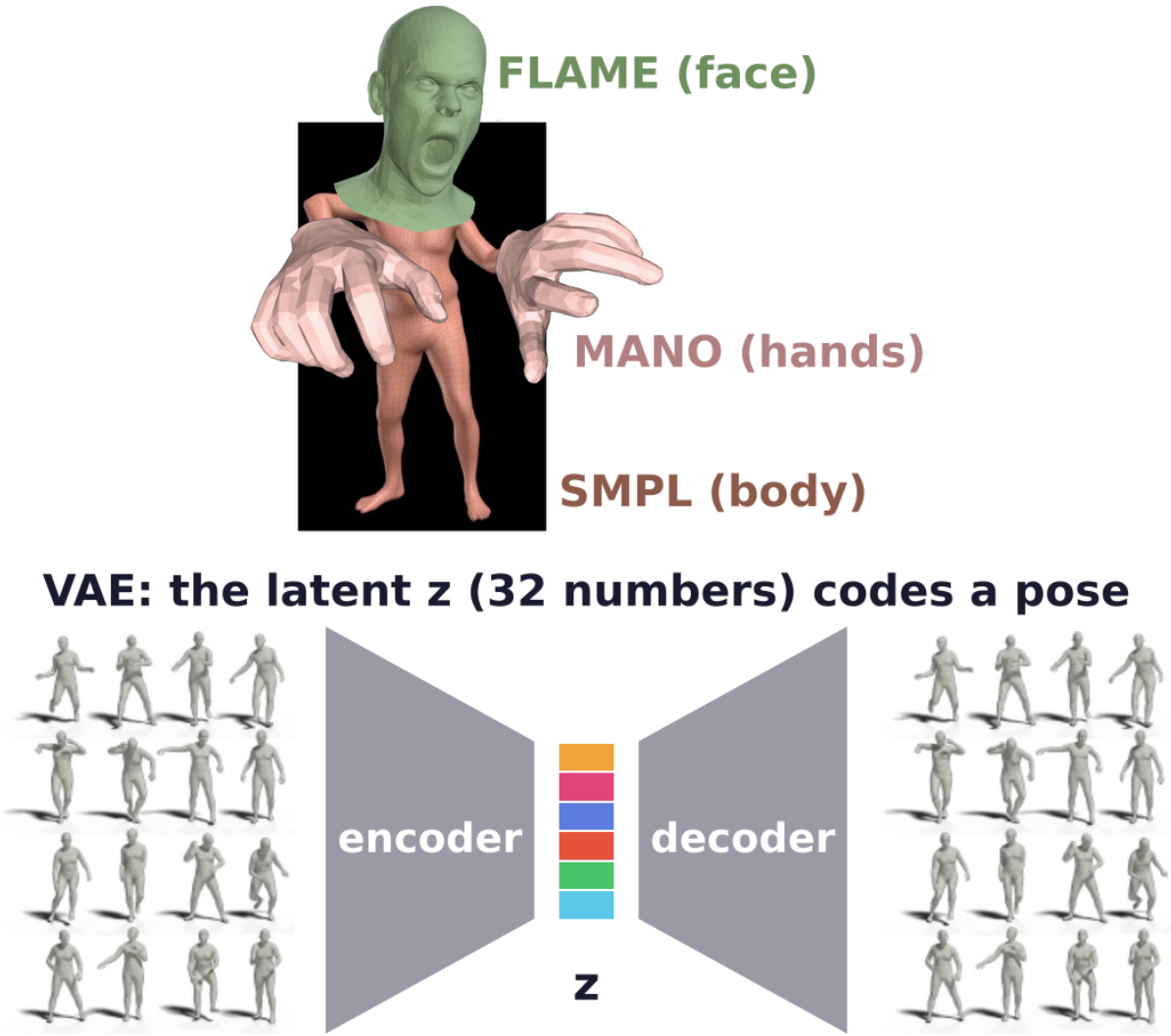


Figure 20: SMPL-X combines the FLAME face, the MANO hands and the SMPL body (P p. 46). The VAE encodes poses into a latent z of 32 numbers (drawn schematically) and decodes them back into poses (P p. 47).

Local optimization can be fast but needs initialization

Optimization has limitations. When the 2D CNN fails (a limb missed, two people's joints mixed up), the fit follows the wrong joints. When the depth is ambiguous, the fit chooses the wrong depth, as in a dive seen from the side. Local optimization can be fast and accurate, but it is prone to local minima and requires initialization: it finds the minimum nearest to its start. The matching cost itself is ambiguous: several bodies fit the same 2D joints. Can we use learning to address some of these limitations?

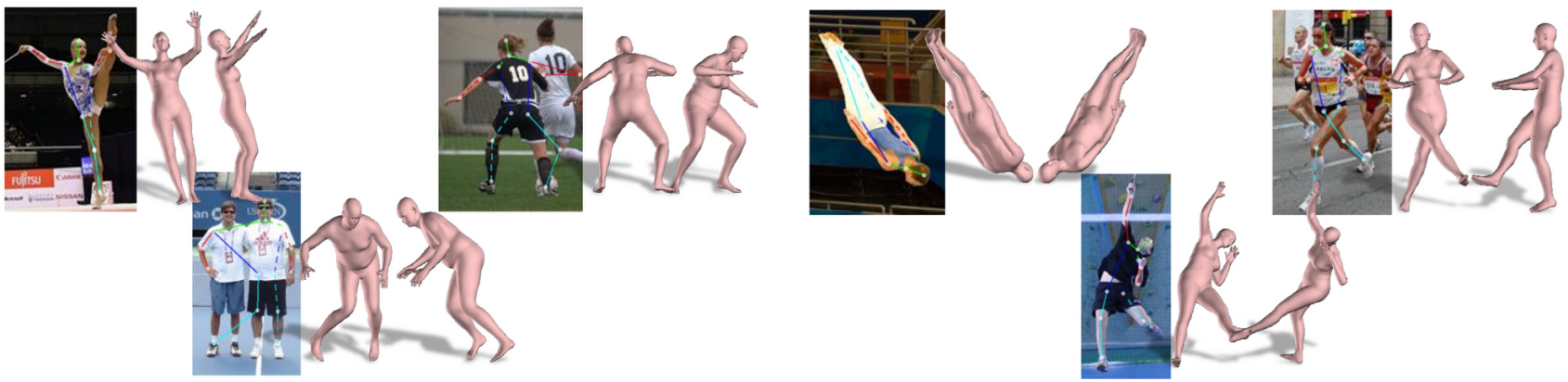


Figure 21: Failures of SMPLify: images (with their 2D joints) and two views of each fit. First three examples: the 2D CNN fails; last three: the depth is wrong. P pp. 50–51.

Humans in 4D: every person's 3D body, tracked over time

Goel's talk answers this question with learning, and moves from images to videos. An image is a 2D (x, y) observation; the state to recover is the 3D (x, y, z) mesh of every person in it. A video is a 3D (x, y, t) observation; its state is 4D (x, y, z, t): every person's mesh over time. 4DHumans (Goel et al., 2023) does both: a network, HMR 2.0 (Human Mesh Recovery 2.0), reconstructs each person in each frame, and a tracker links the reconstructions over time. The network is discriminative and replaces optimization at test time; optimization comes back to produce its training data.

Humans in 4D: every person's 3D body, tracked over time

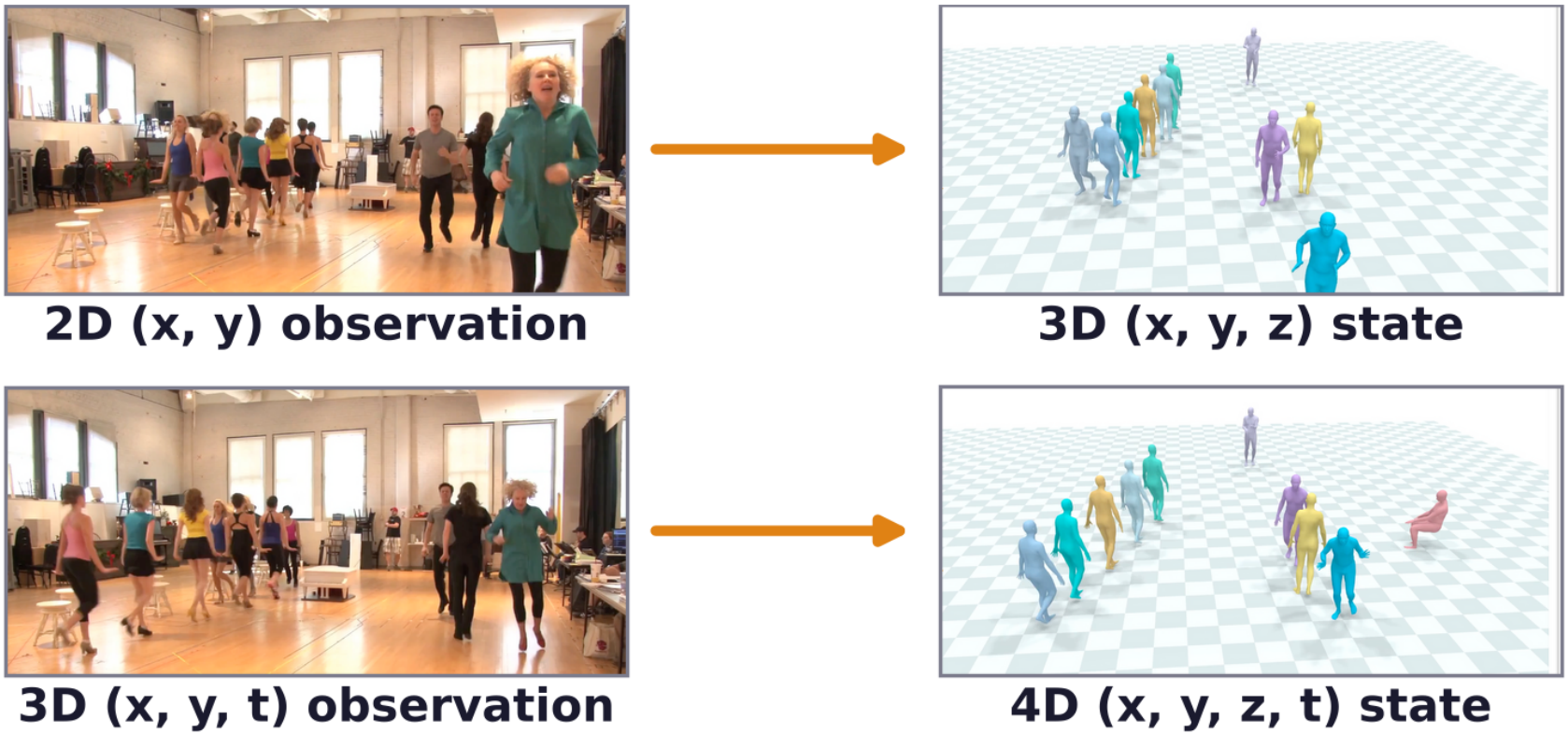


Figure 22: A rehearsal room: one frame and the 3D meshes of its people on a ground plane; the same for a later frame, with each person keeping a color over time. H p. 3.

Human Mesh Recovery (HMR) regresses SMPL parameters

Human Mesh Recovery (HMR; Kanazawa et al., 2018), HMR 2.0's predecessor, is a discriminative model: one forward pass of a network maps an image of a person to the pose $\vec{\theta}$, the shape $\vec{\beta}$ and the camera π, and SMPL turns $\vec{\theta}$ and $\vec{\beta}$ into the mesh. The encoder is a ResNet-50 (a residual network of 50 layers, a standard CNN for images); a regressor reads its features and refines its estimate three times, each time predicting a correction to the previous one. The camera is weak perspective: the 3D joints X are projected orthographically and then scaled by s and shifted by a 2D translation b, $\hat{x} = s\,\Pi(RX) + b$, where R is the global rotation of the body. The losses compare the reprojected 2D joints with labelled ones, compare 3D joints and parameters when they are known, and add discriminators, small networks trained to tell the predicted poses and shapes from real ones from a database of 3D bodies; fooling them acts as a learned prior.

Human Mesh Recovery (HMR) regresses SMPL parameters

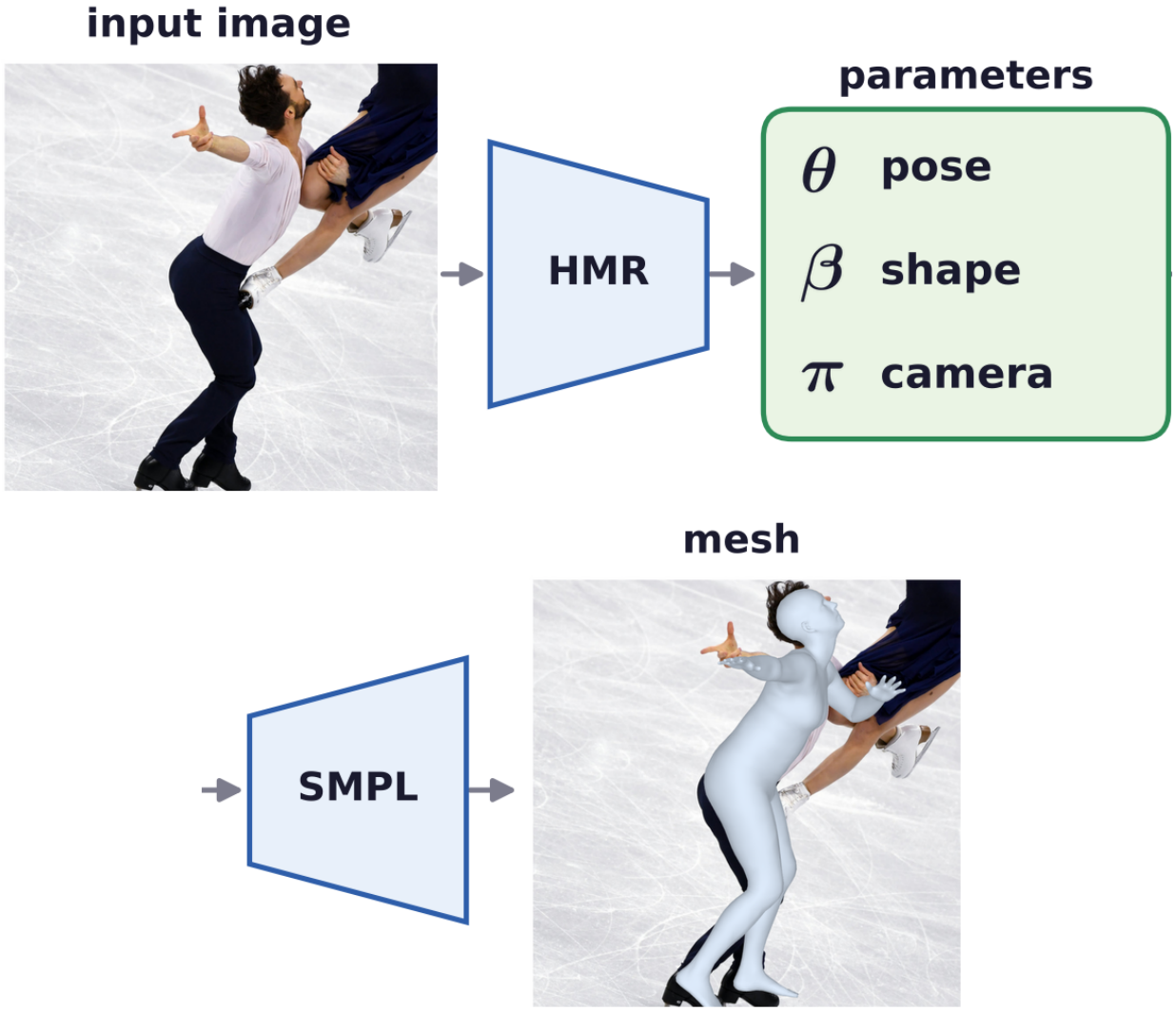


Figure 23: HMR: input image, network, parameters $\vec{\theta}$, $\vec{\beta}$, π. Second row: SMPL turns $\vec{\theta}$ and $\vec{\beta}$ into the mesh, shown over the image. H p. 16.

HMR 2.0: a big model with a vision transformer

HMR 2.0 keeps the task and changes the network to a big transformer. The encoder is ViT-H/16, the huge vision transformer of Dosovitskiy et al. (2021) with 16×16 patches: a 256×192 crop of the person is cut into $16 \times 12 = 192$ patches, each becomes a token (a vector of 1280 numbers), and 32 layers of self-attention, in which every token updates itself from all the others, process them. The ViT is pretrained to find 2D keypoints (ViTPose; Xu et al., 2022), so its tokens already describe the body. A transformer decoder of 6 layers then lets one query token cross-attend to the 192 image tokens: it forms weighted averages of them, with weights computed from its similarity to each. An MLP (multilayer perceptron; in the code, one linear layer) reads $\vec{\theta}$, $\vec{\beta}$ and π from the query's output, with no iterative refinement. The camera is perspective with a fixed focal length, and the network predicts its translation.

HMR 2.0: a big model with a vision transformer

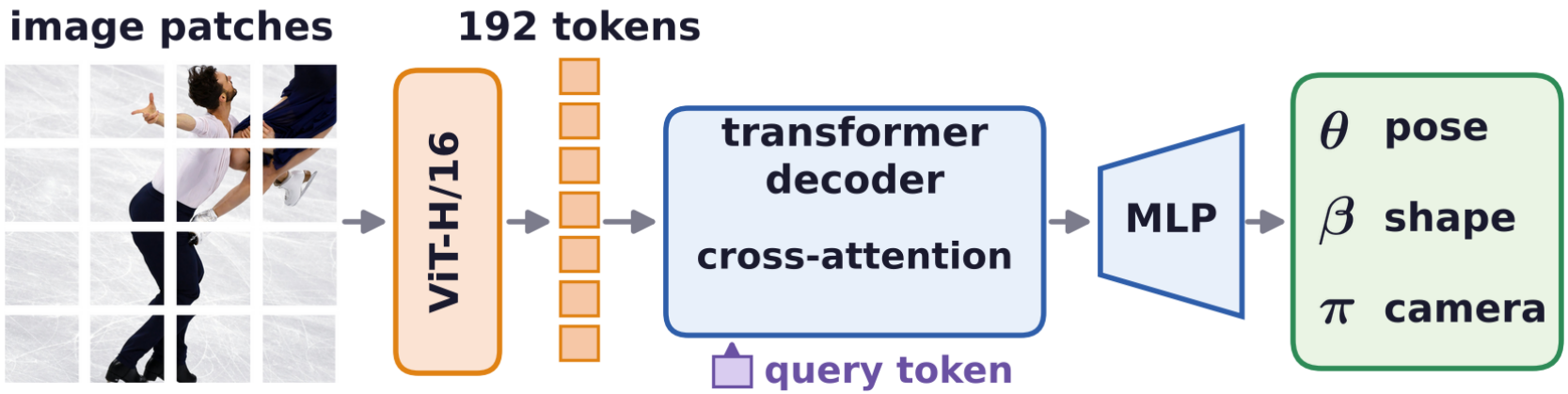


Figure 24: HMR 2.0 (patches and tokens drawn schematically): image patches, the ViT-H/16 encoder, 192 image tokens, a transformer decoder in which one query token cross-attends to the tokens, and an MLP that outputs the pose, shape and camera. H p. 18.

HMR 2.0: big data with pseudo ground truth from fitting

A big model needs big data. Images with 3D ground truth are few and mostly recorded in one room with motion capture (Human3.6M, MPI-INF-3DHP); images in the wild (COCO, MPII) have only 2D keypoints. HMR 2.0 labels unlabelled images by fitting: detect the people (ViTDet, a detector built on a ViT), find their 2D keypoints (ViTPose), and fit SMPL to the keypoints by minimizing a 2D joint term plus priors, as SMPLify does (with the learned pose prior of ProHMR, a probabilistic HMR; Kolotouros et al., 2021). Fits with few keypoints, extreme shapes, implausible poses or large errors are filtered out. The result is pseudo ground truth: 3D labels computed by fitting instead of measured, as in SPIN (Kolotouros et al., 2019), which fits SMPL inside HMR's training loop. HMR 2.0a trains on these four datasets with the ViT frozen (its weights fixed); HMR 2.0b adds pseudo ground truth for frames of the videos of AVA (atomic visual actions) and InstaVariety and the images of AI Challenger, also trains the ViT, and runs ten times as many iterations (with a smaller batch).

HMR 2.0: big data with pseudo ground truth from fitting



Figure 25: An image, its detected 2D keypoints, and (orange arrow) the SMPL mesh fitted to them as pseudo ground truth. The mesh is copied as Goel's slide shows it, standing, unlike the player: a still of the fit. Kolotouros et al. (2019); H p. 19.

The training losses compare joints and add a learned prior

Each training image contributes the losses for which it has labels:

$$\mathcal{L}_{kp3D} = \|X - X^*\|_1, \qquad \mathcal{L}_{kp2D} = \|\pi(X) - x^*\|_1,$$
$$\mathcal{L}_{smpl} = \|\vec{\theta} - \vec{\theta}^*\|_2^2 + \|\vec{\beta} - \vec{\beta}^*\|_2^2, \qquad \mathcal{L}_{adv} = \sum_k \left(D_k(\theta_b, \vec{\beta}) - 1\right)^2.$$

X are the 3D joints of the predicted mesh and $\pi(X)$ their projection by the predicted camera; X^*, x^*, $\vec{\theta}^*$ and $\vec{\beta}^*$ are the labelled 3D joints, 2D keypoints and SMPL parameters (measured or pseudo). $\mathcal{L}_{kp2D}$ plays the role of SMPLify's data term, without confidences or robust penalty, and is minimized over the whole training set instead of for one image. In $\mathcal{L}_{adv}$, θ_b is the body pose without the global rotation, and a discriminator D_k for each factor of the body (each joint's rotation, the whole pose, the shape) scores whether the prediction looks human; the network is trained to push the scores to 1. The weights of the terms (0.05 for $\mathcal{L}_{kp3D}$, 0.01 for $\mathcal{L}_{kp2D}$, 0.0005 for $\mathcal{L}_{adv}$) are in the paper's supplement.

HMR 2.0 recovers unusual poses in the wild

HMR 2.0 recovers unusual poses in the wild: figure skating, ballet, skateboarding, breakdance. Each mesh is predicted from one image, with no smoothing over time. The front view draws the mesh over the image, the view of the camera; the side and top views show the same 3D mesh from other directions, where errors in depth would show.



Figure 26: Two groups of examples. In each group, in order: input image, mesh over the image (front view), side view and top view. H p. 20.

Among these methods, HMR 2.0b is the most accurate in 2D

Three benchmarks compare HMR 2.0 with two earlier regressors, PyMAF-X (Zhang et al., 2023) and PARE (Kocabas et al., 2021). On 3DPW (3D Poses in the Wild), videos with 3D ground truth from body-worn sensors, the error is the mean distance between predicted and true 3D joints in mm (lower is better), either after aligning the pelvis or after Procrustes alignment, which rotates, translates and scales the prediction to fit the truth. On COCO (images) and PoseTrack (videos), which have only 2D labels, PCK@0.05 (percentage of correct keypoints) is the fraction of projected joints that land within 5 % of the image size of their labels (higher is better). HMR 2.0a is the best on 3DPW; HMR 2.0b, trained longer on more data in the wild, is the best in 2D at a cost on 3DPW (81.3 against 70.0 mm; after Procrustes it trails PyMAF-X and PARE). (Goel's slides swap the bars of PyMAF-X and PARE; the figure follows the paper's tables.)

Among these methods, HMR 2.0b is the most accurate in 2D

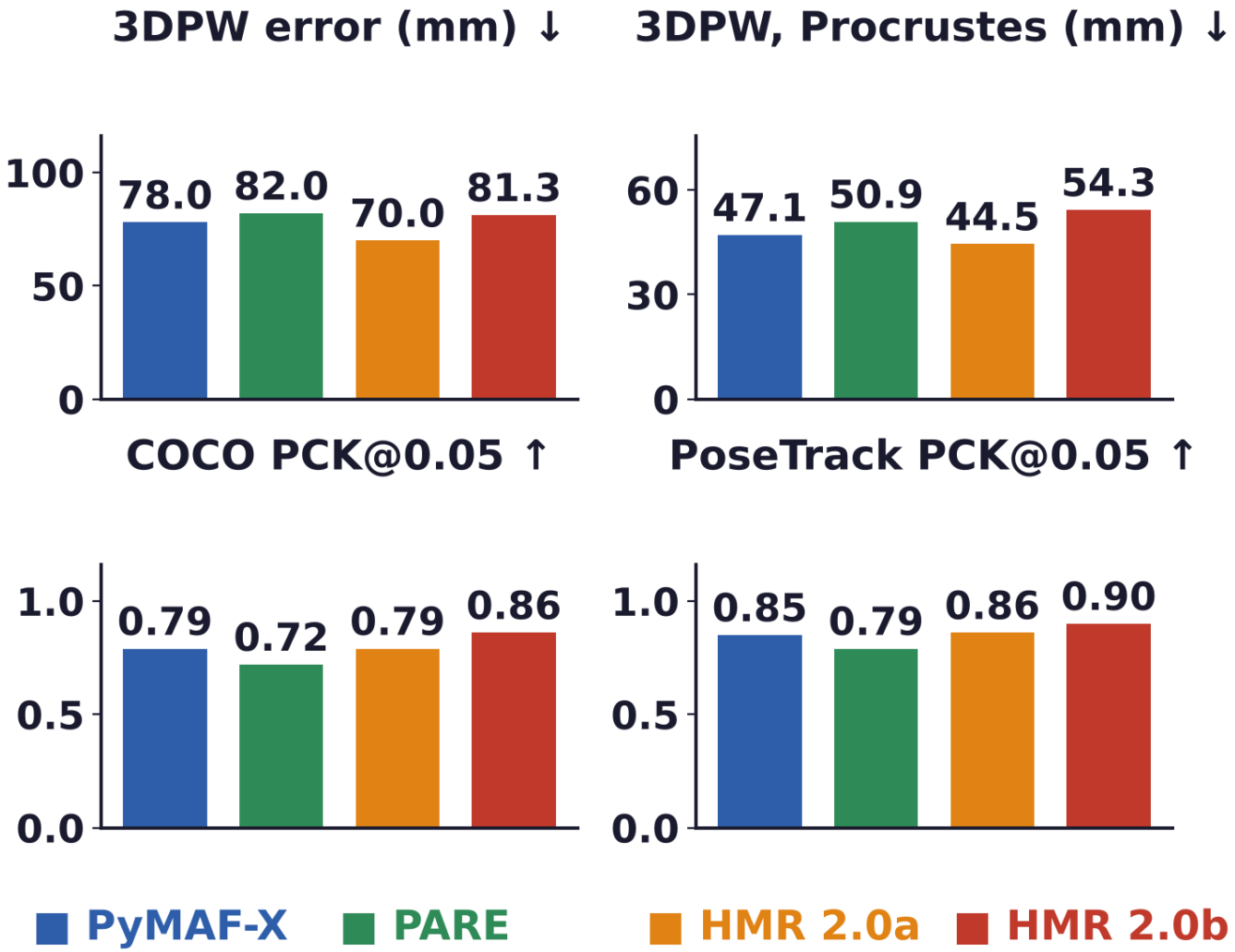


Figure 27: 3DPW error without and with Procrustes alignment (mm, lower is better), and PCK@0.05 on COCO and PoseTrack (higher is better), for PyMAF-X, PARE, HMR 2.0a and HMR 2.0b. Goel et al. (2023), Tables 1–2; H pp. 26–31.

From 3D to 4D: tracking associates people across frames

To go from 3D to 4D, the people detected in each frame must be associated with the people already tracked. The tracker builds on PHALP (predicting human appearance, location and pose; Rajasegaran et al., 2022). Each detected person (a red box) is lifted to 3D by HMR 2.0, and each track keeps three attributes: the SMPL pose, the 3D location, and the appearance, the colors of the image sampled onto the visible part of the mesh. Each track predicts its attributes one step ahead (a transformer predicts the pose), and each pair of track and detection gets a cost that adds $-\log P$ of each cue:

$$\Phi_C = -\log P_A - \log P_P - \log P_{XY} - \log P_N,$$

where P_A, P_P, P_{XY} and P_N are the probabilities that the detection has the predicted appearance (A), pose (P), image location (XY) and depth (N, from the paper's notation). The Hungarian algorithm then finds the assignment of detections to tracks with the smallest total cost.

From 3D to 4D: tracking associates people across frames



Figure 28: Two frames with detected people (red boxes), their meshes from HMR 2.0 seen from the side, and the association of each person across the frames. H p. 33.

4DHumans tracks people better than earlier trackers

PoseTrack contains videos of many people with labelled boxes and identities. HOTA (higher order tracking accuracy) combines how well people are detected with how well their identities are kept; MOTA (multiple object tracking accuracy) counts missed people, false detections and identity switches; IDF1 is the F1 score of identities over whole tracks; IDs counts identity switches (lower is better). With the same person detector (Mask R-CNN) as PHALP and its predecessor T3DP, 4DHumans has 22 % fewer identity switches than PHALP, $(541 - 421)/541$; with the ViTDet detector it improves further.

PoseTrack	HOTA ↑	IDs ↓	MOTA ↑	IDF1 ↑
Trackformer	46.7	1263	33.7	64.0
Tracktor	38.5	702	42.4	65.2
AlphaPose	37.6	2220	36.9	66.9
Pose Flow	38.0	1047	15.4	64.2
T3DP	50.6	655	55.8	73.4
PHALP	52.9	541	58.9	76.4
4DHumans	**54.3**	**421**	**59.8**	**77.9**
4DHumans + ViTDet	57.8	382	61.4	79.1

Goel et al. (2023), Table 4; H p. 39.

3D poses and tracks improve action recognition

AVA (Gu et al., 2018) labels every person's actions in movie clips. LART (Lagrangian action recognition with tracking; Rajasegaran et al., 2023) gives a transformer each person's PHALP track (3D pose and location) and appearance features. It scores 42.3 mAP (mean average precision over action classes), against 39.5 for the best earlier video model.



Figure 29: Tracked people in a hurdles race; each action is scored separately. H p. 40.

4DHumans places people in the camera's coordinate frame

Are we done? The meshes of 4DHumans are placed in the camera's coordinate frame: their positions are relative to the camera. When the camera moves, for instance when it pans along a row of rowers, its motion is added to the motion of the people, who then appear to move when they do not. Placing people in the world requires the camera's own motion, its scale (the camera's motion in metres) and the ground.

4DHumans places people in the camera's coordinate frame

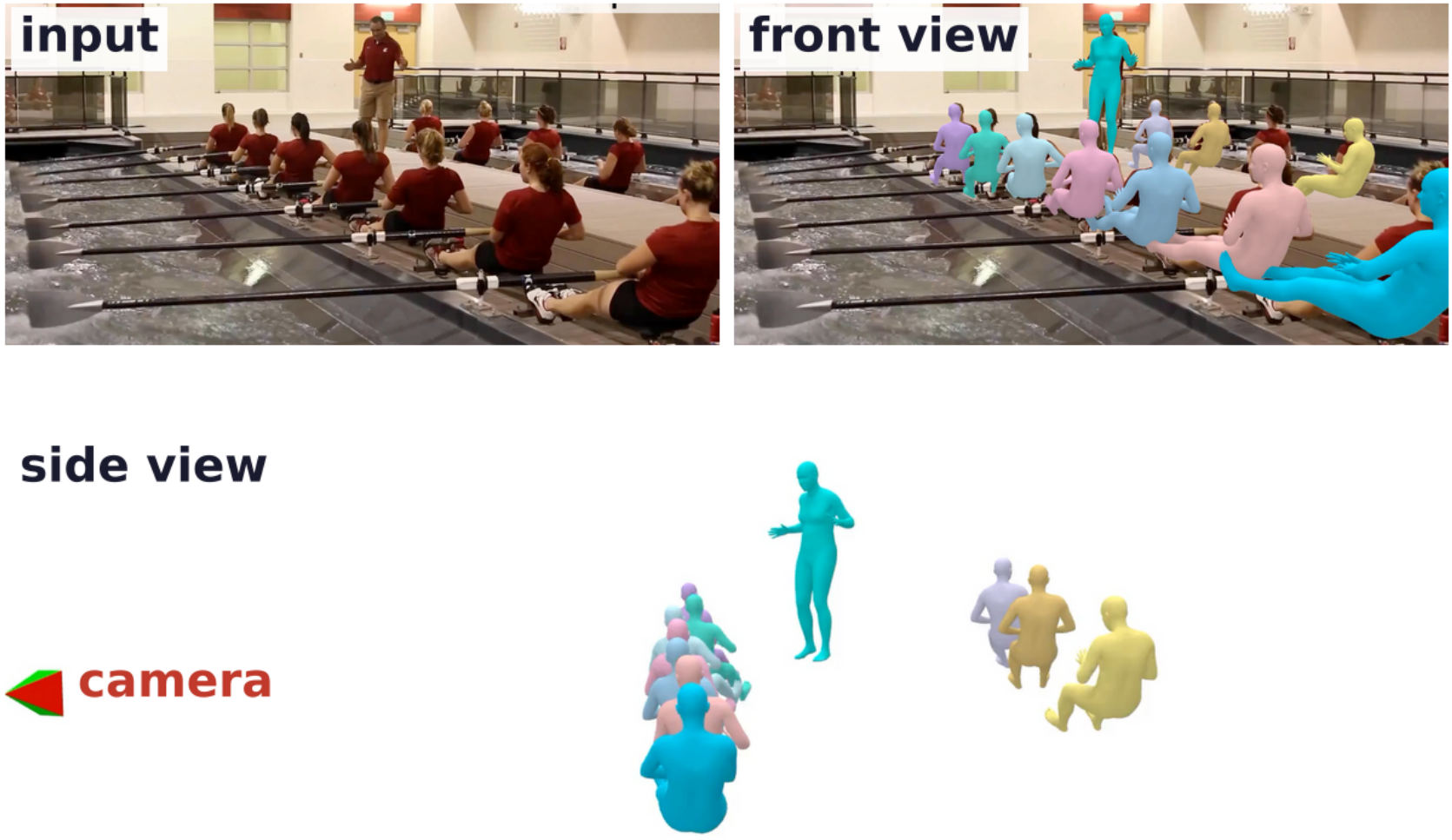


Figure 30: Rowers training indoors: the input, the meshes over it (front view), and a side view of the meshes with the camera (red and green cone). As the camera pans, these meshes move with it though the rowers stay in place. H p. 44.

SLAHMR decouples human and camera motion in the wild

SLAHMR (Ye et al., 2023), named after SLAM (simultaneous localization and mapping) and HMR, separates the two motions. A learned SLAM system (DROID-SLAM) estimates the camera's motion up to an unknown scale α. The people's world trajectories are then optimized: a robust 2D joint term as in SMPLify, with the moving camera scaled by α, pose and shape priors, and HuMoR, a learned prior on how human motion evolves. All people share one world frame, the scale and a ground plane; several people constrain the scale better than one. On EgoBody (videos with 3D ground truth) the world-frame joint error falls from 387.8 mm for the per-frame tracks (PHALP+) to 141.1 mm.

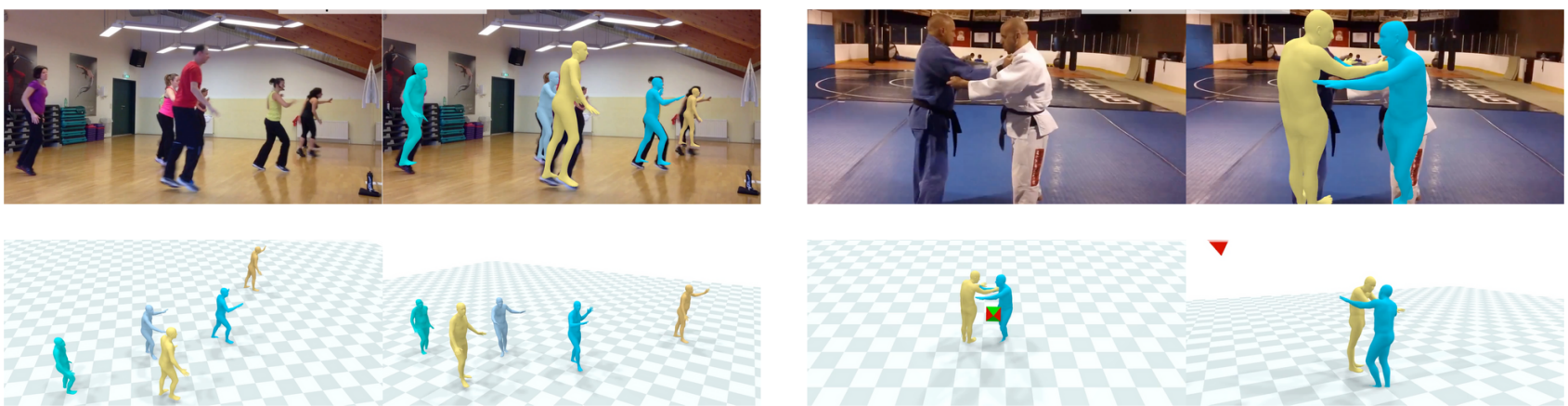


Figure 31: Two examples: input and meshes over the image; under them, two views of the people on the recovered ground (red marks: camera). Ye et al. (2023); H pp. 47, 49.

Summary

- 3D pose and shape from one image are ambiguous; SMPL describes a person by a few parameters.
- SMPL deforms a template with shape and pose blend shapes and poses it by linear blend skinning.
- Fitting a generative model: extract features, predict and match them, optimize $p(x \mid I) \propto p(I \mid x)\, p(x)$.
- Non-linear least squares: Gauss-Newton linearizes the residuals; Levenberg-Marquardt damps the step.
- SMPLify fits SMPL to 2D joints with pose, shape and interpenetration priors: a MAP estimate.
- Optimization needs an initialization; a network can predict the parameters directly from the image.
- HMR 2.0: a vision transformer regresses SMPL; big data gets pseudo ground truth from fitting.
- 4DHumans tracks the 3D people over time; SLAHMR separates their motion from the camera's.

References

[1] G. Pons-Moll. Fitting SMPL to images with optimization. Lecture 6_1, Virtual Humans, University of Tübingen, winter 2023/24 (P).

[2] S. Goel. Humans in 4D. Talk slides, 2023 (H).

[3] C. J. Taylor. Reconstruction of articulated objects from point correspondences in a single uncalibrated image. *CVIU*, 80(3):349–363, 2000.

[4] C. Sminchisescu and B. Triggs. Covariance scaled sampling for monocular 3D body tracking. *CVPR*, I:447–454, 2001.

[5] M. Loper, N. Mahmood, J. Romero, G. Pons-Moll and M. J. Black. SMPL: a skinned multi-person linear model. *ACM Trans. Graphics (Proc. SIGGRAPH Asia)*, 34(6):248, 2015.

[6] C. Sminchisescu. Consistency and coupling in human model likelihoods. *Proc. IEEE Int. Conf. Automatic Face and Gesture Recognition*, 27–32, 2002.

[7] G. Pons-Moll, A. Baak, T. Helten, M. Müller, H.-P. Seidel and B. Rosenhahn. Multisensor-fusion for 3D full-body human motion capture. *CVPR*, 663–670, 2010.

References

[8] K. Levenberg. A method for the solution of certain non-linear problems in least squares. *Quarterly of Applied Mathematics*, 2(2):164–168, 1944.

[9] D. W. Marquardt. An algorithm for least-squares estimation of nonlinear parameters. *J. SIAM*, 11(2):431–441, 1963.

[10] F. Bogo, A. Kanazawa, C. Lassner, P. Gehler, J. Romero and M. J. Black. Keep it SMPL: automatic estimation of 3D human pose and shape from a single image. *ECCV*, LNCS 9909, 561–578, 2016.

[11] L. Pishchulin, E. Insafutdinov, S. Tang, B. Andres, M. Andriluka, P. V. Gehler and B. Schiele. DeepCut: joint subset partition and labeling for multi person pose estimation. *CVPR*, 4929–4937, 2016.

[12] P. Guan, A. Weiss, A. O. Bălan and M. J. Black. Estimating human shape and pose from a single image. *ICCV*, 1381–1388, 2009.

[13] G. Pavlakos, V. Choutas, N. Ghorbani, T. Bolkart, A. A. A. Osman, D. Tzionas and M. J. Black. Expressive body capture: 3D hands, face, and body from a single image. *CVPR*, 10975–10985, 2019.

References

[14] A. Kanazawa, M. J. Black, D. W. Jacobs and J. Malik. End-to-end recovery of human shape and pose. *CVPR*, 7122–7131, 2018.

[15] S. Goel, G. Pavlakos, J. Rajasegaran, A. Kanazawa and J. Malik. Humans in 4D: reconstructing and tracking humans with transformers. *ICCV*, 14783–14794, 2023.

[16] A. Dosovitskiy et al. An image is worth 16x16 words: transformers for image recognition at scale. *ICLR*, 2021.

[17] Y. Xu, J. Zhang, Q. Zhang and D. Tao. ViTPose: simple vision transformer baselines for human pose estimation. *NeurIPS*, 38571–38584, 2022.

[18] N. Kolotouros, G. Pavlakos, D. Jayaraman and K. Daniilidis. Probabilistic modeling for human mesh recovery. *ICCV*, 11605–11614, 2021.

[19] N. Kolotouros, G. Pavlakos, M. J. Black and K. Daniilidis. Learning to reconstruct 3D human pose and shape via model-fitting in the loop. *ICCV*, 2252–2261, 2019.

References

[20] H. Zhang, Y. Tian, Y. Zhang, M. Li, L. An, Z. Sun and Y. Liu. PyMAF-X: towards well-aligned full-body model regression from monocular images. *IEEE TPAMI*, 45(10):12287–12303, 2023.

[21] M. Kocabas, C.-H. P. Huang, O. Hilliges and M. J. Black. PARE: part attention regressor for 3D human body estimation. *ICCV*, 11127–11137, 2021.

[22] J. Rajasegaran, G. Pavlakos, A. Kanazawa and J. Malik. Tracking people by predicting 3D appearance, location and pose. *CVPR*, 2740–2749, 2022.

[23] C. Gu et al. AVA: a video dataset of spatio-temporally localized atomic visual actions. *CVPR*, 6047–6056, 2018.

[24] J. Rajasegaran, G. Pavlakos, A. Kanazawa, C. Feichtenhofer and J. Malik. On the benefits of 3D pose and tracking for human action recognition. *CVPR*, 640–649, 2023.

[25] V. Ye, G. Pavlakos, J. Malik and A. Kanazawa. Decoupling human and camera motion from videos in the wild. *CVPR*, 21222–21232, 2023.